第八届全球校园人工智能算法精英大赛（AIC） · 算法应用赛道

哈基米（Hajimi）· LXH 智慧养老护理服务机器人

技 术 报 告

赛题 14 · **智慧养老护理**

平台：carebot 三轮全向服务机器人（STM32 下位机 + 镭神 M10 + Astra S + 讯飞六麦）

环境：Ubuntu 16.04 + ROS Kinetic + Python 2.7

代码：**7 个自研 ROS 功能包**（34 个 Python 文件，16 211 行）

加 69 个标定 / 诊断 / 部署脚本（10 465 行）

最完整一轮：**205.8 s 零失败**，18 / 18 段导航全部成功

2026 年 10 月

本报告全部数据来自 2026-10-21 至 2026-10-22 的实车运行日志，
对应代码基线 tag **V100**；文中给出的每个数字都能在 docs/logs 的日志里找到对应行。

摘要

本作品面向独居老人居家照护中“出事没人知道”这一最现实的困境，在组委会提供的 carebot 三轮全向移动平台上，实现了一条可无人干预跑完的居家巡视与照护链路：读取二维码确定的房间巡视顺序 → 播报客厅光照与温湿度 → 在 A 点完成主人身份核验 → 按序巡视书房、厨房、卧室并完成漏水检测、烟雾检测与风扇控制、电灯与窗帘控制 → 回到 A 点进行三轮语音交互 → 返回随机起点并复位家电。场地为 3 m × 3 m 的模拟居室，全程无人工遥控。

软件以四层结构组织（决策、能力、导航、驱动）。最上层是任务编排层，把十二个步骤串成带超时、带降级、可重入的状态机；其下是五个自研能力节点，分别负责二维码识别、人脸核验、物联网桥接、语音交互与命名点位导航；再往下是组委会提供的 AMCL + move_base 导航栈，本项目按窄场地特征重写了代价地图、恢复行为与到达判定；最底层是雷达、相机、底盘与麦克风阵列驱动，不做改动。全部推理在本机完成，代码中不存在把图像发往设备的路径。

技术路线上有三个关键取舍。其一是“宽容取景、严格判读”：识别端用画面下部 ROI、CLAHE 增强、二倍上采样与三帧一致去抖把二维码读出来，判读端只接受 {2,3,4} 的严格排列，宁可重试也不接受可疑顺序。其二是“多帧投票”的人脸门禁：单帧置信度不足为凭，改为五秒窗口内的帧级投票，命中率与平均置信度同时达标才放行。其三是“命名点位 + 容差锁定”的窄场地导航：把 18 个带语义的航点写进配置，以 0.30 m 到达容差配合 latch 判定，允许倒车脱困，并用自研的卡住检测与“后退—转向—重发”恢复替代整圈旋转。

实测结果：最完整一轮全流程用时 205.8 s、零失败项，18 段导航全部 SUCCEEDED，单段到达误差 0.148–0.289 m、多数在 0.16 m 量级；二维码在第一圈 0.2 s 内命中，转圈期间定点保持偏移 0.0 cm；人脸核验 12/12 帧命中；厨房风扇与卧室电灯均完成“下发—回读—确认”闭环；物联网全量刷新往返约 0.7 s；全栈从冷启动到可发目标 15 s。

关键词： 服务机器人；ROS Kinetic；AMCL；DWA；二维码识别；LBPH 人脸核验；HomeAssistant；离线命令词；状态机

目录

摘要 2
第一部分 作品概述 4
1.1问题从哪来 4
1.2现有设备为什么没被用起来 4
1.3目标人群与场景 5
1.4作品构成与主要性能指标 6
1.5主要创新点 6
第二部分 需求分析 8
2.1赛题约束与三处随机 8
2.2功能需求与验收标准 9
2.3非功能需求与量化指标 10
2.4与组委会参考实现的差距 10
第三部分 系统设计 11
3.1总体架构 11
3.2分层职责与包划分 12
3.3一次任务从头到尾 12
3.4坐标系与关键接口 13
3.5启动与自检 15
3.6场地与命名航点 15
第四部分 关键技术 18
4.1二维码：读得宽容，判得严格 18
4.2人脸核验：多帧投票是必需的 19
4.3窄场地导航：让车别卡在门口 20
4.4航向基准：一个 97.5 度的系统偏差 22
4.5激光滤波：一次花了两天的排查 22
4.6物联网：双通道与写后回读 23
4.7语音交互：离线命令词 + 意图匹配 + 三轮对话 25
4.8任务编排：每一步都有退路 27
4.9没有显卡时的算法选型 27
第五部分 实测与验证 28
5.1测试方法 28
5.2一次完整实车运行 29
5.3导航精度与耗时 29
5.4识别与核验可靠性 32
5.5时延与启动 32
5.6重复性 32
第六部分 评分点对照与能力证据 34
6.1逐项对照赛题评分表 34
6.2每条得分的证据链 34
6.3面向后续接入的接口设计 35
第七部分 可靠性与失效分析 36
7.1方法 36
7.2失效模式清单 36
7.3风险矩阵 37
7.4三项闭环是怎么做的 37
7.5安全联锁 38
第八部分 人文、安全与伦理 39
8.1数据不出户 39
8.2采集最小化 39
8.3面向感官退化的交互 39
8.4防误触发 40
8.5我们明确不做的事 40
8.6人机边界 40
第九部分 工程实践与版本演进 41
9.1赛前预检 41
9.2仿真回归 41
9.3复盘与报告自证 41
9.4现场部署流程 42
9.5版本演进 42
9.6工程纪律 42
第十部分 结论 44
参考文献 45
附录 A 术语与缩略语 46
附录 B 代码规模 47
附录 C 关键配置索引 48
附录 D 一次运行的时间线（节选） 49

第一部分 作品概述

1.1 问题从哪来

晚上十点，一位独居老人起夜去卫生间，地砖上有水，脚下一滑。摔倒本身未必致命，真正致命的是接下来那几个小时——没有人经过，没有人听见，手机还留在卧室床头柜上。

这类事件有一个共同结构：事情本身不大，无人知晓才致命。燃气忘了关、水龙头没拧紧、夜里起身头晕、药忘了吃，单看都是小事，但只要没人及时知道，就会滚成大事。

我们要回答的问题很朴素：一台成本可控的移动平台，能不能替家属完成“每天过去看一眼”这件事？不需要它做诊断，也不需要它替代护理人员，只要能按时走一圈、看到该看的、说该说的、遇到不对的喊一声，就已经解决了很多家庭最现实的焦虑。

1.2 现有设备为什么没被用起来

市面上并不缺监测设备。跌倒手环、烟雾报警器、门窗传感器、云摄像头，单品都不贵，电商平台几百块就能配齐。但真正走进老人家里、并被长期使用的并不多。

第一个原因是误报。烟雾报警器在厨房煎鱼时响，人体传感器因为宠物经过而触发，摄像头因为窗帘飘动推送一条“检测到移动”。年轻人还能分辨，老人不行。报警响到第三次，最理性的选择就是把电池抠掉。

第二个原因是隐私。摄像头装在客厅甚至卧室门口，画面持续上传到某个服务器。老人未必说得清“数据主权”，但他们清楚地知道“不想被人一直看着”，而这种不适一旦产生，很难用功能弥补。

第三个原因更朴素——不会用。智能家居 App 动辄三四层菜单，字体小、图标抽象，注册要手机号、登录要验证码。对一位七十多岁、眼睛老花、手指不太灵活的人来说，这套流程本身就是一道墙。

三条放在一起看，指向同一个结论：养老设备真正的门槛不在技术能力，而在“能不能被持续使用”。本作品的设计因此从一开始就围绕少误报、不出户、不用学这三件事展开。

1.3 目标人群与场景

把“老人”当成一个整体是危险的，不同自理能力的人，需要的帮助完全不同。我们把目标人群拆成三类，并据此决定优先做什么。

表 1-1　目标人群细分与功能定位

人群	日常处境	最担心的事	本作品的定位
A　高龄独居、基本自理	子女不在身边，起居靠自己，反应与记忆在退	忘关电器、夜里起身、忘吃药	**核心覆盖**：定时巡视 + 家电联动 + 语音提醒
B　失能或半失能	长期卧床或行动不便，需定时探视	跌倒后无人知晓、饮水服药无人协助	覆盖巡视与环境播报，告警链路可扩展
C　白天独处、与子女同住	白天家里没人，晚上有人照料	白天出状况没人发现、电器空转	**核心覆盖**：巡视与环境控制

A 与 C 的痛点是“看得见、够得着”的——机器人走过去、报个环境数据、把灯关掉，事情就解决了，而且能在 3 m × 3 m 的场地里完整验证。B 类最需要的是 7 × 24 的连续看护，本作品通过巡视频次与语音交互把“被发现”的时间窗压到一次巡视之内，并在系统结构上为后续的告警接入预留了统一接口。

1.4 作品构成与主要性能指标

作品由软件与硬件两部分构成。硬件沿用组委会提供的 carebot 平台，不改动任何结构；软件是 7 个自研 ROS 功能包（34 个 Python 文件、16 211 行），加上 69 个标定、诊断、部署与仿真脚本（10 465 行）。

表 1-2 作品主要性能指标

指标	当前实现 / 参数	说明
主控与运行环境	Intel i7-7500U（2 核 4 线程）/ 7.7 GB / 无独立显卡；Ubuntu 16.04 + ROS Kinetic + Python 2.7	无 GPU 是本方案全部算法选型的前提
底盘与运动	三轮全向，STM32F407 下位机，RS485 轮毂电机；导航限速 0.22 m/s，允许 −0.10 m/s 倒车	支持横向平移，3 m 窄场地少掉头
感知配置	镭神 M10 激光雷达 10 Hz；Astra S 深度相机 30 fps；AHRS IMU 200 Hz；讯飞六麦阵列	全部由组委会提供，本项目只做配置与标定
定位与导航	AMCL（500 – 2000 粒子，omni 模型）+ move_base（GlobalPlanner + DWA）；到达容差 0.30 m 且 latch；18 个命名航点	窄场地专项参数，见 4.3
识别能力	二维码：画面下部 ROI + CLAHE + 2× 上采样 + pyzbar + 三帧一致去抖；人脸：Haar 检测 + LBPH 特征 + 多帧投票	均为传统视觉方法，可全程本地运行
交互与联动	讯飞离线命令词（12 条指令）+ 在线 TTS；HomeAssistant REST 主通道 + 串口网关备份	零屏幕依赖，全部走语音
代码规模	7 个包 34 个 Python 文件 16 211 行；含工具脚本共 103 个文件 26 676 行	另有 15 个 launch、64 份配置
系统时延	冷启动到可发目标 15 s；物联网全量刷新往返约 0.7 s；单段导航 0.7 – 32 s	见 5.3
全流程成绩	205.8 s 零失败，18 / 18 段导航 SUCCEEDED，到达误差 0.148 – 0.289 m	2026-10-22 实车，见 5.2

1.5 主要创新点

（1）全本地推理架构。把“隐私”做成架构约束而非协议承诺：识别、规划、决策全部在本机完成，图像只在内核与进程内存之间流转，代码中不存在把图像发往设备的路径，断网仍可完整跑完任务。

（2）宽容取景、严格判读的双端设计。识别端尽量“多试”（ROI、增强、上采样、多帧去抖、原地旋转扫描），判读端一步不让（只接受 {2,3,4} 的严格排列）。把“降误报”做成算法机制，而不是靠调一个阈值。

（3）多帧投票的人脸门禁。不以单帧置信度判定，而以五秒窗口内的帧级投票（有效帧数、命中率、平均置信度三条同时满足）决定放行，并在不通过时换角度重采。

（4）窄场地的“命名点位 + 容差锁定 + 自研恢复”导航策略。18 个带语义的航点代替一串裸坐标；0.30 m 容差配合 latch 判定，让“站到那儿”取代“精确对齐”；允许倒车、去掉整圈旋转，改用“后退—转向—重发”的定向恢复。

（5）写后回读的物联网闭环。每一条控制指令都跟随一次状态回读，回读窗口按设备单独配置；实体编号与逻辑名分离，重新配网只需改一张表。

（6）全程自证的时间线与日志。任务每一步都写入结构化的时间线，结束时自动汇总总耗时与失败项；报告里出现的每个数字都能在日志里找到对应行。

第二部分　需求分析

2.1 赛题约束与三处随机

赛题给了一个场景、一条任务链和一堆限制条件。我们的做法是先把限制读透，因为限制往往比功能更能决定架构。

任务链本身并不复杂：开机、去二维码区、读顺序、报环境、核验身份、按序巡视三个房间、回起点。但规则里埋了三处随机，每一处都会让“把坐标写死”的做法失效。

第一处是起点随机。机器人每次被放在场地开口处的不同位置，方向也不定。我们的做法是：任务开始前把车放正，启动脚本连发 5 次初始位姿估计（0,0,0，协方差 0.25/0.25/0.0685）把 AMCL 拉回地图原点，之后每一步都用“相对当前定位”发目标；回程则用固定的起点航点，配合 0.30 m 到达容差与 latch 判定，因此起点在 0.3 m 范围内偏移都不会影响“回起点”这一项的完成判定。

第二处是二维码位置随机，现场公布。我们没法提前把“二维码前两米”标成固定点位，只能把 `qr_center` 标成“那个区域里一个能看到墙的可达点”，再靠识别环节去补偿位置误差：先走到观察点，再原地低速旋转扫描，最多两圈，首圈没读到就后退 0.15 m 换视角再来一圈。

第三处是巡视顺序编码在二维码里。这条最要命——顺序读错，后面三个房间的任务点全走错，直接影响 30 分。所以判读环节的可靠性比速度重要得多：容忍识别端的各种不确定性，但对“顺序”本身零容忍。

规则里还有容易被忽略的一条：一分钟内未对情景作出回应即判故障。这句话听起来是裁判规则，落到工程上就是硬约束：状态机每一步都必须有超时，而且超时之后必须有兜底动作，绝不能死等。我们给导航段 120 s、二维码 15 s / 两圈、人脸核验 3 轮、物联网刷新 3 次重试，任何一步超时都走“降级继续”而不是“卡住”。

2.2 功能需求与验收标准

把上面的约束翻译成可验收的功能条目，共十二条。每一条都对应一个可复现的验收方式，且都能在实车日志里找到证据。

表 2-1 功能需求与实现状态

编号	功能需求	验收标准	实现模块	状态
FR-01	开机后自主导航启动	从静止到接受目标点，无遥控介入；全栈冷启动 ≤ 30 s	carebot_nav_kit + m40 起栈	已实现
FR-02	识别二维码给出的巡视顺序	200 mm 码、1.0–2.0 m 距离、连续 3 帧一致	carebot_qr	已实现
FR-03	播报客厅光照、温度、湿度	三项齐全；缺项时降级播报已取得的部分	carebot_iot + carebot_voice	已实现
FR-04	A 点完成主人身份核验	五秒窗口内多帧投票，命中率与置信度双阈值同时满足	carebot_face	已实现
FR-05	核验不通过不放行	播报原因并重采，最多 3 轮	face + 任务编排	已实现
FR-06	按二维码顺序巡视三房间	顺序与码一致；单房间失败不影响后续	任务编排	已实现
FR-07	卧室开电灯 + 开窗帘	下发后回读 on / open，失败重试	carebot_iot	已实现
FR-08	厨房读烟雾 + 开风扇	播报烟雾状态；风扇回读确认	carebot_iot	已实现
FR-09	书房漏水检测并播报	读水浸传感器，播报“正常 / 异常”	carebot_iot	已实现
FR-10	返回随机起点	用固定起点航点 + 容差 latch，误差 ≤ 0.30 m	carebot_nav_kit	已实现
FR-11	全程语音讲解与三轮语音交互	每步主动播报；三轮任意任务，结束语即收尾	carebot_voice	已实现
FR-12	服务缺失时降级运行	关键服务不可用则播报并走降级，不死等	全线	已实现

2.3 非功能需求与量化指标

对养老场景而言，非功能需求往往比功能需求更能决定产品是否真的被使用。我们把五条非功能需求量化成可测量的指标，并在实现中逐条落实。

表 2-2 非功能需求与量化指标

编号	需求	量化指标	实现手段 / 实测
NFR1	低误报	人脸须多帧一致且命中率与置信度双阈值同时达标；二维码须为 {2,3,4} 的严格排列	多帧投票 + 严格归一化校验；实车连续两轮零误判
NFR2	隐私	上传云端的数据量 = 0 字节	全本地推理，图像仅内存处理；唯一网络调用为局域网内的 HA 状态读写
NFR3	简单交互	老人操作步骤 ⩽ 1 步	语音唤醒 + 14 条固定命令词，词表已落地并现场验证
NFR4	实时性	单段导航 ⩽ 45 s（平均）；IoT 刷新 ⩽ 1 s；全流程 ⩽ 300 s	实测：最大段 32.3 s、IoT 约 0.7 s、整轮 205.8 s
NFR5	鲁棒性	任一环节失败不得死等	每阶段强制超时 + 重试上限 + 降级路径；卡住自动恢复

2.4 与组委会参考实现的差距

组委会给了 40 余个 ROS 包作为参考平台，但它只解决了“车能跑起来”，不解决赛题。我们逐项比对了覆盖度，把缺口直接转化为功能清单——这也是本项目七个自研包的分工依据。

表 2-3 与参考实现的覆盖度对比

能力	参考实现	本作品	处理方式
二维码识别	完全缺失（全工作区检索不到相关代码）	自研 carebot_qr	从零开发，15 分且决定后续顺序
风扇 / 窗帘控制	未覆盖	已实现，带回读校验	补进设备映射与回读闭环
漏水 / 烟雾	未覆盖	已实现	自建传感器映射与阈值判定
人脸与任务联动	有人脸节点，无门禁判定	新增 face_gate	多帧投票 + 核验结果参与任务决策
HomeAssistant 桥	有，但令牌硬编码、字符串切 JSON、回调里死循环	重写 iot_bridge	JSON 解析 + 令牌注入 + 定时器轮询 + 回读
任务状态机	通用巡逻 smach 示例，与赛题无关	自研任务编排	按十二步流程重写
导航参数	差速模型、local costmap 用 map 坐标系、拼写错误	修正为 omni + rolling window	见第三、四部分
语音交互	有识别节点，无词表、无多轮对话	carebot_voice + 对话状态机	BNF 词表 + 意图匹配 + 三轮对话

第三部分 系统设计

3.1 总体架构

组委会提供的平台能跑，但不会自己完成赛题——它缺一个“知道该干什么”的大脑，也缺几个“能把事情干成”的能力。我们做的是在决策与能力这两层上补东西，并把已有的导航栈按窄场地特征重新标定。

系统按五层组织：最底下是硬件；其上是组委会提供的驱动层，本项目不改；再上是导航栈，复用但重写参数与恢复策略；其上是五个自研能力节点，各自负责一件事；最上面是任务编排，只有一个节点，但它决定整条链路怎么走。

L4 决策层	**任务编排**（m36_mission_v5.py，1 207 行） 十二步状态机 • 顺序规划 • 超时与降级 • 时间线日志 由 `m40_fast_start.sh` 在栈就绪后自动拉起，全程无人工介入
L3 能力层	**carebot_qr** 二维码识别 • **carebot_face** 人脸核验 • **carebot_iot** 物联网桥接 • **carebot_voice** 语音交互 • **carebot_nav_kit** 命名点位导航与自检
L2 导航层	AMCL 粒子滤波（omni 模型） • move_base（GlobalPlanner + DWA） • local / global costmap • laser_filter 激光滤波 • odom_imu 里程计融合
L1 驱动层	lslidar_driver • astra_camera • base_control（stm32） • art_imu • xf_mic_asr_offline • xf_tts —— 组委会提供，本项目不修改
L0 硬件层	STM32F407 三轮全向底盘 • 镭神 M10 激光雷达 • Astra S 深度相机 • 讯飞六麦阵列 • HomeAssistant 盒子 + Zigbee 传感/执行器

图 3-1 系统分层架构。L0 / L1 由组委会提供，L2 复用并重新标定，L3 / L4 为本项目自研

3.2 分层职责与包划分

这样切分有个实际好处：出问题时能快速定位是“能力坏了”还是“编排错了”。比如机器人到了房间却没开灯，先看 `/iot/control` 服务有没有被调用——调用了没生效，是物联网层的问题；根本没调用，是状态机的问题。两者排查方向完全不同。

表 3-1 七个自研功能包

功能包	行数	核心文件（行数）	职责
carebot_eldercare	3 773	mission_manager.py（1 272） preflight_check.py（653） build_report.py（459） rehearsal.py（443）、sim_stubs.py（417）	可安装版任务编排：状态机、结构化事件日志（JSONL）、赛前预检、复盘与报告生成、仿真桩
carebot_nav_kit	9 256	waypoint_nav.py（671） nav_check.py（713） fake_sensors.py（802） odom_monitor.py（367） set_initial_pose.py（294）	命名点位导航、初始位姿注入、激光滤波配置、里程计监控、无硬件仿真场地
carebot_qr	744	qr_detector.py（744）	二维码识别与宽容解析：6 种载荷格式、严格排列校验、去抖
carebot_face	607	face_gate.py（607）	Haar 检测 + LBPH 识别 + 五秒窗口多帧投票的门禁判定
carebot_iot	1 278	iot_bridge.py（786） iot_serial_bridge.py（492）	HomeAssistant REST 主通道 + 串口网关备份；写后回读校验
carebot_voice	553	voice_bridge.py（553）	语音话题归一化、播报队列、TTS 合成与播放、唤醒角发布
carebot_iot_msgs	—	Control.srv（7）	带参设备控制服务类型：device / action / extra

3.3 一次任务从头到尾

任务编排有两套实现，共用同一组服务接口：一套是安装在 `carebot_eldercare` 里的“可安装版”，带预检、复盘与报告生成，日常在仿真里做回归；另一套是实车用的单文件版本（`m36_mission_v5.py`），好处是启动快、改一处不用重新编译工作空间，赛场上用这一套。两者的阶段划分、超时与降级策略完全一致，下面以实车版本为准。

任务编排节点是整个系统的大脑，一千二百多行。它不直接驱动硬件，而是把每一步翻译成对能力节点的服务调用，同时把过程写成结构化时间线。

开机后先自检。move_base 是硬依赖，起不来就直接终止；雷达、相机、物联网这三项缺了只记告警，因为缺了也能勉强跑，只是效果打折。自检通过后先做一次设备在线检查——卧室灯、卧室窗帘、厨房风扇三个可控设备必须都不处于掉线状态，最多等 30 s，确认之后才播“开始执行任务”。这一步把“设备没上线”从“跑到厨房才发现”提前到了“发车之前”。

接着导航到二维码区，原地低速旋转扫描。识别成功后顺序先在本节点解析一遍，拿不到再回退到默认顺序，同时把这次回退记成警告。然后是客厅环境播报。

接下来去 A 点做人脸核验。这一步是门槛，规则写的是“识别到主人才可进入房间”，所以核验环节采用多帧投票，不通过就换角度重采，最多三轮。

核验通过后进入巡视循环，按二维码给的顺序逐间走。每到一个房间先播报到达，再执行该房间的动作并回读确认。全部走完回到 A 点做三轮语音交互，最后回起点、关掉风扇与电灯，报总耗时、写日志收尾。

① 起点自检 设备在线检查 → 播报“开始执行任务”	**② 客厅入口** 导航至 living_in（0.42, −0.90）
③ 二维码扫描 旋转扫描 → 三帧一致 → {2,3,4} 严格校验	**④ 顺序解析** 兼容 234 / 123 / BCD；失败回退默认顺序并告警
⑤ 环境播报 光照 / 温度 / 湿度，缺项降级播报	**⑥ 人脸核验** A_mid → A 点，五秒窗口多帧投票
⑦a 语音预热 停相机、后台起 ASR，为 A 点交互做准备	**⑦ 三房间巡视** 书房漏水 / 厨房烟雾 + 风扇 / 卧室灯 + 窗帘
⑦b 语音交互 回 A 点，三轮任意任务，说“没什么事”即收尾	**⑧ 返回起点** 先回客厅再回起点；关风扇、关电灯

图 3-2 一次任务的十个编排阶段（导航段共 18 段，其中房间往返被拆成多个命名航点）

3.4 坐标系与关键接口

坐标关系直接决定了代价地图与激光滤波阈值的取值，因此单独列出。车体碰撞半径 0.18 m，costmap 取 0.21 m；雷达相对车体前伸约 8 cm，这 8 cm 是后面 4.5 节那场排查的根源。

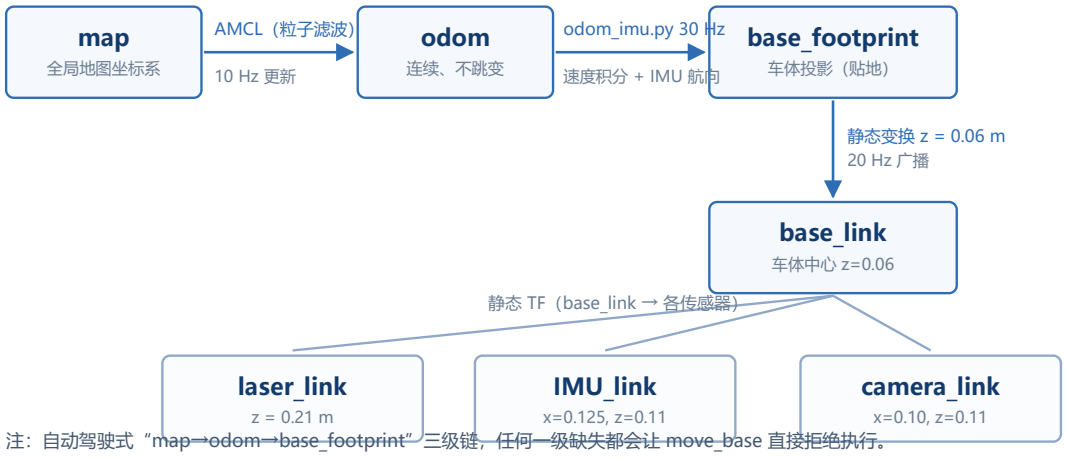


图 3-3 坐标变换树与各段发布频率

四类接口贯穿全流程：话题用于持续状态（如环境数据、识别结果），服务用于一次性动作（如触发一次识别、控制一台设备），“命名点位”接口把坐标封装成语义，任务层不再关心坐标从哪来。

compact

表 3-2 对外主要接口

接口	类型	方向	说明
/qr/detect_once	std_srvs/Trigger	服务	触发一次识别，返回 “2,3,4|原始载荷”
/qr/room_order	Int16MultiArray	发布	解析出的房间顺序（latch，供任务层兜底）
/face/verify	std_srvs/Trigger	服务	执行一次五秒窗口核验，返回判定与明细
/face/verified	std_msgs/Bool	发布	核验结果（latch）
/iot/refresh	std_srvs/Trigger	服务	全量刷新环境与设备状态，返回 JSON
/iot/control	carebot_iot_msgs/Control	服务	带参设备控制：device / action / extra
/iot/env	std_msgs/String	发布	结构化环境数据（JSON，2 s 周期）
/voice/say	std_msgs/String	发布	播报文本，由语音节点合成并播放
/voice_words	std_msgs/String	发布	离线命令词的识别结果（UTF-8）
/nav/goto_named	自定义服务	服务	按命名点位发目标，内部完成坐标换算
/move_base	move_base_msgs/MoveBaseAction	动作	导航执行接口

3.5 启动与自检

现场只给两分钟准备时间，所以“起得来”本身就是一项能力。我们把起栈做成全并行：清场、起 master、所有节点同时拉起，用“就绪判据”而不是固定 sleep 决定何时开跑。

表 3-3 冷启动流程与就绪判据

阶段	动作	就绪判据	实测
1	清理残留（9 类进程 + 相机节点，避免抢 USB）	进程计数归零，相机设备释放	2 s
2	起 roscore（独立端口 11399，与其他组隔离）	端口可连接	1 s
3	并行拉起 15 个节点/launch：雷达、底盘、IMU、3 组静态 TF、里程计、激光滤波、地图、相机、AMCL、move_base、二维码、人脸、物联网、TTS	8 份参数文件随起随灌，无需等待	—
4	就绪判定	5 个话题有数据（相机 12 s、其余 6 s）→ 连发 5 次初始位姿 → TF `map→base_footprint` 通 → 3 个服务在线 → move_base action 在线；总截止 24 s	3.6 s
5	底盘驱动检查	`/cmd_vel` 的订阅者包含 `/stm32_node`，否则重启底盘并复检 8 s	0.5 s
6	自动发起任务	任务进程拉起，日志落到 `/tmp/m36_run7.log`	—

实测冷启动到“可以发目标”为 15 s（脚本目标 ≤ 30 s）。这 15 s 里包含 2 s 清场与 3.6 s 就绪等待，其余为并行拉起的时间。

3.6 场地与命名航点

场地是 3 m × 3 m 的模拟居室，中间一道隔断把客厅与三个房间分开，门洞净宽约 0.9 m。我们把整条路线离散成 18 个带语义的命名航点，每个名字对应“在这个位置要做什么”，而不是一串裸坐标——这既让状态机的可读性提升，也让现场改点位时只改一张表。

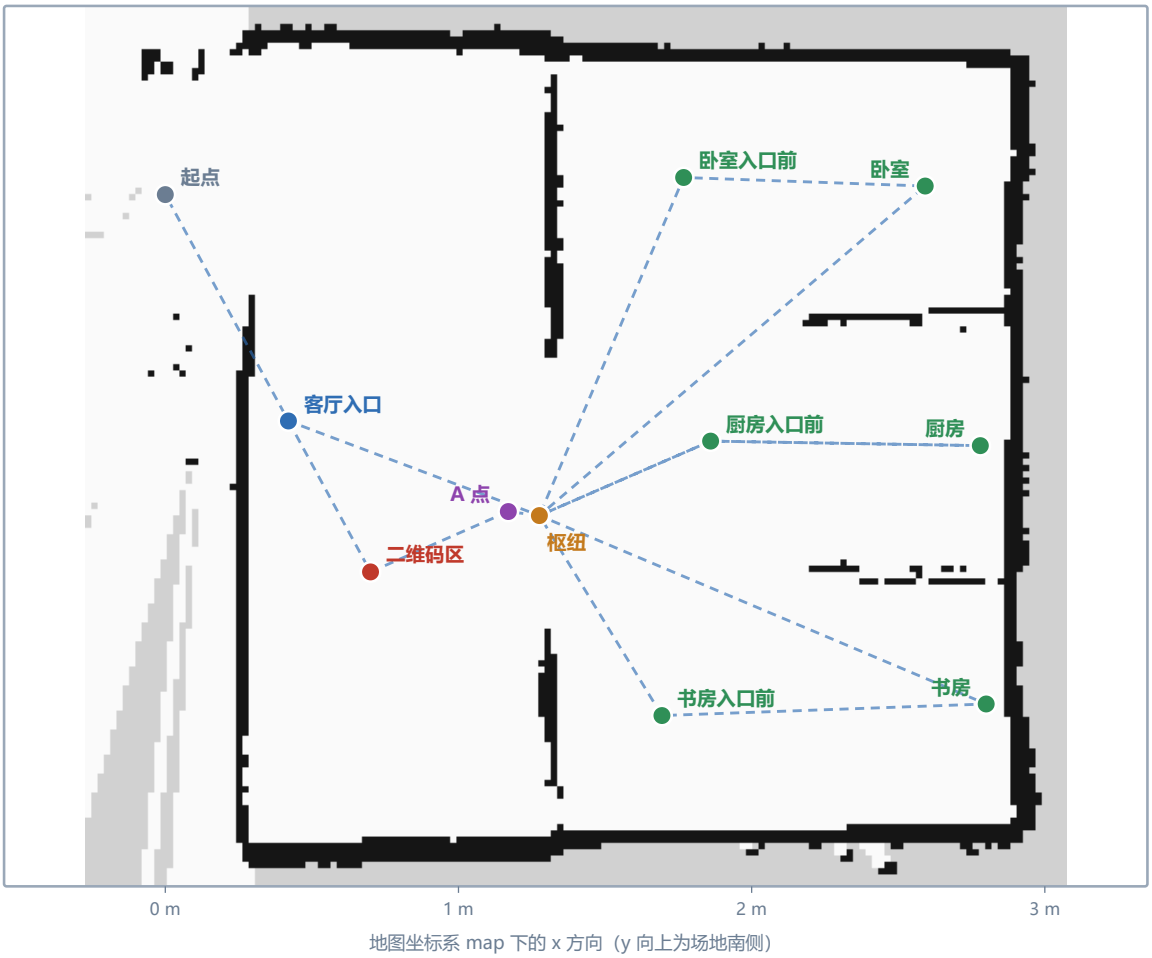


图 3-4 实车建图结果与 18 个命名航点（虚线为巡视路线示意）

compact

表 3-4 关键命名航点（map 坐标系，单位 m）

航点	坐标 (x, y)	朝向	用途
start	(0.000, 0.000)	保持车头	起点 / 终点（随机起点按 0.30 m 容差处理）
living_in	(0.420, −0.900)	保持车头	客厅入口，去二维码区与返程的中转点
qr_center	(0.700, −1.500)	保持车头	二维码观察点（旋转扫描的中心）
A / A_mid	(1.170, −1.260) / (0.780, −0.960)	行进方向	人脸核验点 / 进 A 点的过渡点
hub	(1.276, −1.276)	行进方向	三房间共用的枢纽，所有往返都经过它
B_door / B	(1.694, −2.071) / (2.800, −2.025)	行进方向	书房入口前 / 书房内（漏水检测）
E_C / C_door / C	(1.860, −0.980) / (1.827, −1.047) / (2.780, −0.998)	行进方向	厨房门外 / 门口 / 房间内（烟雾 + 风扇）
D_door / D	(1.768, 0.068) / (2.592, 0.034)	行进方向	卧室入口前 / 卧室（电灯 + 窗帘）

第四部分 关键技术

这一部分讲六个关键问题：怎么把二维码读对、怎么确认来人是谁、车为什么在门口磨蹭、机器人为什么往反方向跑、激光为什么要按几何关系设阈值，以及没有显卡时算法怎么选。前两节是识别，后三节是导航与定位，最后一节是选型依据。

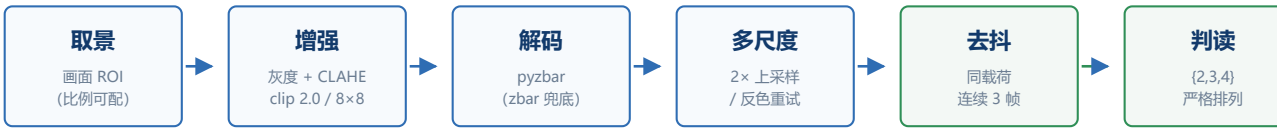
4.1 二维码：读得宽容，判得严格

二维码占十五分，而且决定后面三十分房间任务的顺序。这个项目一开始就有一个矛盾：现场光照不确定、码的位置不确定、距离不确定，识别端必须足够宽容；但识别结果一旦错了，后面的代价是三十倍。

我们的处理是把宽容和严格分给两个环节。前面尽可能多地尝试：默认只取画面下部六成五的区域（现场按码位高度可调为全幅）、灰度化、做一次限制对比度自适应直方图均衡（CLAHE，clip 2.0，8 × 8 网格），用 pyzbar 解码；一次解不出就做二倍立方上采样再解，仍失败就把图像反色再试一遍。同一帧里出现多张码时，按“面积占 0.6 + 居中程度占 0.4”的加权分挑最可信的那一张。

载荷解析同样宽容：既接受 `2,3,4` 这种纯数字串，也接受 JSON（`order` / `rooms` / `room_order` / `sequence` 等键或裸数组）、URL 查询参数（`order=` / `rooms=` / `seq=`）、`B,C,D` 字母形式，以及中文房间名。编码先按 UTF-8 解，失败再按 GBK 解——现场打码用的是哪套编码，我们事先不知道。

后面则一步不让。解析结果必须恰好是二、三、四这三个数字的一个排列，数量不对、有重复、出现别的值，一律判为失败。失败时把原始载荷用语音念出来，并回退到默认巡视顺序，同时把这次回退记成警告——顺序可以猜，但不能错得没有提示。



若 3 帧内未得到稳定结果：原地旋转扫描（0.28 rad/s、最多 2 圈，带定点保持）→ 后退 0.15 m 换视角再转一圈

若载荷稳定但判读失败：念出原始载荷并回退默认顺序，绝不带着可疑顺序往下走

对外：/qr/detect_once（服务）、/qr/room_order（Int16MultiArray，latch）

图 4-1 二维码识别与判读链路

单帧结果不可信，所以我们加了帧级去抖：同一个载荷必须连续三帧一致才被接受。为了在没有稳定视角时也能读到，任务层还实现了一个“旋转扫描”阶段：以 0.28 rad/s 原地缓慢旋转，最多两圈，旋

转期间用 P 控制做定点保持（kp = 0.8，限速 0.04 m/s），实测两圈内最大偏离 0.4 cm；首圈没读到就往后退 0.15 m，换一个视角再来一圈。

表 4-1　二维码处理链路

环节	做法	为什么这样做
取景	可配置 ROI（默认画面下部 65%，现场按码位高度调为全幅）	码一般贴得低，上半部分全是墙面与干扰
增强	灰度 + CLAHE（clip 2.0，8 × 8）	场地可能偏暗，远处码分辨率不够
解码	pyzbar 优先，zbar 兜底；6 种载荷格式、UTF-8 / GBK 双编码	现场才知道码的内容与编码方式，必须宽容
多尺度	2× 立方上采样重解；仍失败则反色重解；同帧多码按面积 + 居中加权选优	距离远、反光、颜色相反的码都能救回来
去抖	同一载荷连续 3 帧一致	单帧受反光与运动模糊影响大
判读	必须恰 3 项且集合等于 {2,3,4}	顺序错一次，三个房间的任务分全丢
兜底	旋转扫描（0.28 rad/s、≤ 2 圈、定点保持）；后退 0.15 m 换视角；失败则念原文并回退默认顺序	把“读不到”变成“多试几次”，把“读错了”变成“不会发生”

实车结果：最完整一轮在旋转 3°、0.2 s 时即命中二维码，随后三帧一致成立，判读得到 `{2,3,4}`（书房 → 厨房 → 卧室）；旋转期间定点保持的最大偏移为 0.0 cm。第二圈兜底路径在联调阶段验证可用，但正式轮次没有用上。

4.2 人脸核验：多帧投票是必需的

组委会给了现成的识别节点，用的是 Haar 加 LBPH 这套传统方法。它能识别人脸，但不解决我们的问题——我们要的不是“检测到一张脸”，而是“这个人是不是主人”。

起初用的是单帧判定：置信度过阈值就通过。实际测下来很不安心，同一个人的同一张脸，在不同帧里置信度能差出二十多个点，一次误判就等于把门打开了。后来改成多帧投票：取五秒窗口、5 Hz 采样，只采纳一秒内的新鲜结果，把“有脸帧”与“命中主人帧”分别计数。

判据有两条，必须同时满足：一是命中率（命中主人帧 / 有脸帧）不低于阈值，二是这些命中帧的平均置信度不低于阈值。不通过不是直接放弃，而是原地小幅转个角度重新采，最多三轮。置信度在这里的定义是 `100 - LBPH 距离`，越大越可信；距离大于 100 直接判为“未知人”，不再参与投票。

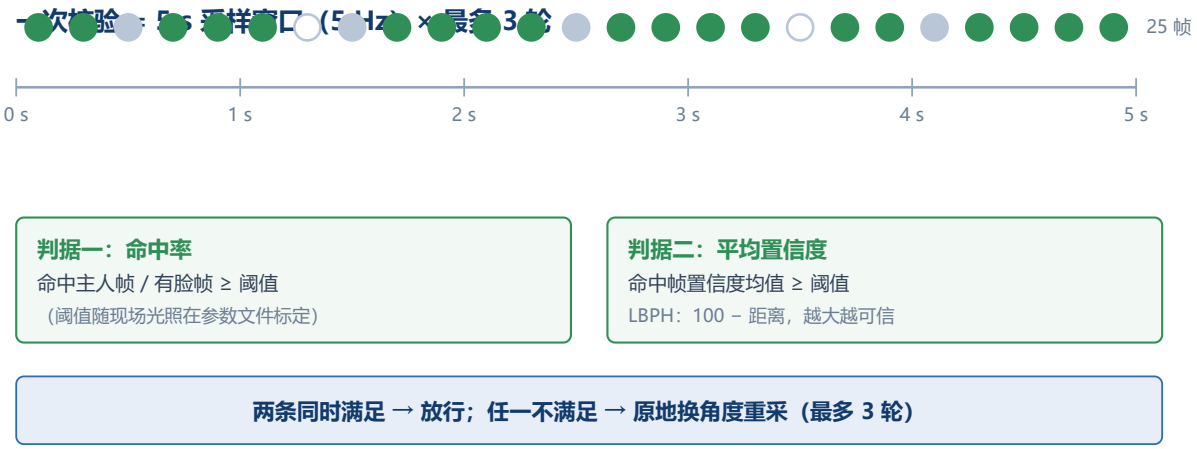


图 4-2 五秒窗口的多帧投票机制（实心为命中主人，灰色为有脸非主人，空心为无脸）

这套机制的价值不在“阈值高低”，而在“判据形式”：单帧的置信度是一个随机变量，多帧的命中率是一个统计量。把判定建立在统计量上，才能让“同一个人换一帧”不再改变结论。

表 4-2 人脸核验的关键参数

参数	取值	说明
检测器	Haar 级联（frontalface_default）	CPU 上单帧毫秒级，无需 GPU
检测参数	scaleFactor 1.2，minNeighbors 5，最小尺寸为画面 10%	兼顾远处与近处的人脸
识别器	LBPH（OpenCV 3.2）	特征模板存 `trainer.yml`，不含原始照片
置信度口径	100 − LBPH 距离，距离 ≥ 100 判未知	数值越大越可信，便于对老人解释
投票窗口	5 s，5 Hz 采样，最多 3 轮	25 帧量级，足够形成稳定统计量
样本与训练	每人 100 张 / 300 s 采集，统一缩放 115 px 后训练	现场采集脚本一键完成

实车结果：12 帧有效采样中 12 帧全部命中主人，命中率 100%，平均置信度达到判定阈值，系统判定通过并播报“身份核验通过，欢迎回家”。

4.3 窄场地导航：让车别卡在门口

导航用了组委会的 AMCL + move_base（GlobalPlanner + DWA），但直接拿默认参数在 3 m 见方的场地上跑不通。最典型的现象是：机器人明明已经站在房间门口，却左右蹭、前进一点退一点，最后要么超时，要么干脆原地转起圈来。

拆开看是三个矛盾挤在一起。其一，目标容差设得太紧（0.08 m），为了凑这个精度，局部规划器会在最后半米反复微调；其二，底盘虽然全向，但配置里不允许后退，一旦进了死角只能靠转向脱身；其

三，move_base 默认带一个“原地转三百六十度”的恢复行为，转完并不保证能算出路，白花十秒，还把定位带飘。

对应做了四处调整，都写进参数文件并留了注释。

表 4-3 窄场地导航的四项调整

参数	原来	现在	原因
目标容差	0.08 m	0.30 m + latch	不再为凑精确位置在门口来回微调；进圈即判定到达
航向容差	0.10 rad	3.14 rad	到房间只要“站在那儿”，不必对齐朝向
最小速度	0.0	−0.10 m/s	窄房间掉头靠倒车脱困
恢复行为	含 rotate_recovery	只保留两项 clearing	整圈旋转无效，改用“后退 + 转向 + 重发”

最不直观的是把航向容差放到 3.14 rad，等于完全不检查朝向。理由很朴素：服务机器人走进房间，重要的是“站到那个位置”，不是“面朝哪个方向”。要求它精确对齐，反而会在窄空间里多出好几次转向动作。与之配套的是“到点朝向 = 行进方向”：发目标时若不指定朝向，就取 `atan2(dy, dx)` 作为默认朝向，车停下来的姿态天然就是“走进来的方向”，既不用额外转身，视觉上也不突兀。

恢复策略我们改成自己的：判卡住的条件是“10 s 内位移小于 0.04 m 且线速度、角速度都接近零”，一旦成立就后退 0.20 m、原地转 ±40°（两次交替方向），然后重发目标，最多三次。这个动作比整圈旋转小得多，在 0.9 m 的门洞前不会把自己转进墙里。

表 4-4 导航与代价地图关键参数（nav_v4）

参数	取值	说明
全局规划器	GlobalPlanner	静态地图上的 A* / Dijkstra 网格规划
局部规划器	DWAPlannerROS	全向模型，允许横向平移
速度约束	0.22 / −0.10 m/s，±0.8 rad/s	窄场地低速，倒车仅用于脱困
加速度	1.5 m/s²，4.0 rad/s²	短距离多，加速度直接决定段耗时
DWA 采样	sim_time 1.5 s，20 × 20 采样	加长预判，减少“无解”
代价权重	path 50 / goal 40 / occdist 0.4	贴合路径优先，障碍代价适度
车体半径	0.21 m	取 0.21 而非 0.25：0.85 m 房间里 0.25 会让 DWA 无解
膨胀半径	0.30 m，cost_scaling 4.0	略大于半径，靠 DWA 避让而不是封死门洞
局部代价地图	4 × 4 m，0.05 m 分辨率，rolling	只保留车周围窗口，静态层 + 障碍层 + 膨胀层
AMCL	500–2000 粒子，beams 60，likelihood_field	3 m 场地不需要更多粒子，省算力

4.4 航向基准：一个 97.5 度的系统偏差

第二个故障更隐蔽。现象是目标在西北，车却往东南开。诡异的是里程计看起来“正常”——数值在动，方向和车头大致一致，只有 AMCL 的估计和激光真值差了半米多。

定位这个问题的办法后来被我们固化成了一个小工具：让车只走十二厘米，同时比较位移方向与车头朝向。长度对、方向不对，说明问题出在坐标系；长度也不对，那才是轮径或编码器的问题。

测出来位移长度 0.114 m（正常），方向 -97.4°，车头朝向 +0.1°，夹角 -97.5°。一个接近一百度的常数量，说明有个航向基准被整体转了一次。

根因在里程计合成那段代码：算位移时用的是 IMU 的绝对航向，而发布朝向时用的是相对航向，两者差一个“启动瞬间的绝对航向”。这台车启动时正好朝 -97.5°，于是位移被整体旋转了将近一百度。里程计说“我在往前走”，实际记录的却是一条横着滑的轨迹，AMCL 拿这个去和激光匹配，自然一路发散。

错的写法

```
dx = v * dt * cos(yaw)          ← 绝对航向
dy = v * dt * sin(yaw)
q  = euler(0, 0, yaw - yaw0)    ← 相对航向（两者不一致）
```

对的写法

```
yaw_rel = normalize(yaw - yaw0)  # 归一化到 ±π
dx = v * dt * cos(yaw_rel)
dy = v * dt * sin(yaw_rel)
q  = euler(0, 0, yaw_rel)        # 位移与朝向用同一个基准
```

改完之后同一测试的夹角从 -97.5° 变成 -0.3°，定位误差收敛到厘米级。这个故障解释了此前所有的怪现象：为什么里程计“看着正常”、为什么换地图调参数都没用、为什么定位总是差半米。现在的里程计节点以 30 Hz 发布 `/odom_clean` 与 TF，航向严格使用相对基准，协方差按实测填写，另有一条 EKF 融合路线（`world_frame=odom`，避免与 AMCL 抢 `map→odom`）作为对照。

4.5 激光滤波：一次花了两天的排查

这是整个项目里最折磨人的故障。现象是导航完全跑不起来：全局规划报“无路径”，局部规划报“原地旋转会碰撞，代价 -1.00”。而所有日志里，没有一句话提到激光雷达。

我们起初怀疑地图、怀疑定位、怀疑参数，挨个换过一遍都没用。真正让方向转过来的是一次偶然观察：把机器人放在空旷处看扫描的原始数据，发现最小值不是零，而是 0.225 m，出现在九十几度的方位上。也就是说，雷达一直在“看见”自己的车体。

想通这一点之后，剩下的就是几何题。雷达装在车体前方约八厘米，所以“雷达到车体外壳”的距离随角度变化：正前方最近只有 0.10 m；正后方最远，0.26 m；侧向 0.18 m。而当时的滤波下限写的是 0.22 m，刚好卡在中间——车头方向的自己滤掉了，车尾方向全漏了。

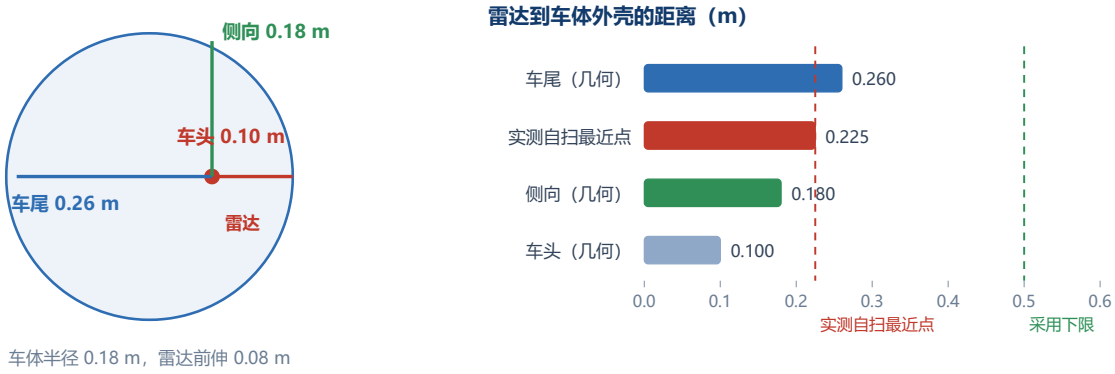


图 4-3 激光“自扫”的几何解释与阈值选取

漏掉的点进了代价地图会怎么样？每帧约一百三十个点、占全部读数的百分之十三，被障碍层当成真实障碍，在机器人周围围出一圈。实测那一圈里有 63 个致命格、两百多个梯度格。于是局部规划器认为机器人贴着一堵墙，连原地转身都判定为碰撞。

表 4-5 激光滤波下限的四组消融

阈值	实测表现	结论
0.22 m	自扫点全部漏过，机器人在假障碍中间，导航完全不可用	不可用
0.30 m	机器人处代价 85 → 0，但仍有零星 aborted	部分可用
0.38 m	仿真可用，实车仍有 0.42–0.52 m 的回波残留	实车不稳
0.50 m	幽灵障碍消失，此前“90 秒只走 0.55 米”的龟速现象一并解决	**采用**

这件事给我们的教训不是“阈值要调大”，而是判据要建立在几何关系上，不能凭经验。正确做法是：把机器人放在周围两米无物的空地，读一遍扫描的最小值，那个数就是“雷达到最远车体点”的距离，阈值取它加八厘米；上限也有讲究，超过“车体最大半径加十五厘米”就会把真实的近处障碍滤掉，那是另一种撞墙。

最终配置是“距离滤波下限 0.50 m、上限 8.0 m（越界置 NaN）+ 斑点滤波（窗口 2 帧、相邻点差 0.1 m）”。这套参数在 V100 那一轮里，18 段导航全程没有再出现幽灵障碍相关的恢复动作。

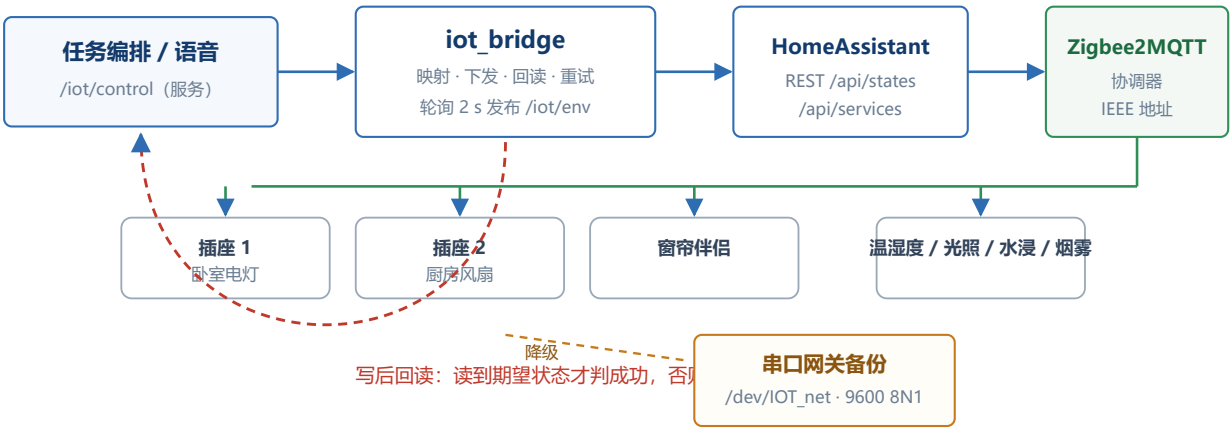
4.6 物联网：双通道与写后回读

家电控制看起来简单，实际是整条链路里最容易“看起来成功、其实没生效”的一环。我们的做法是主备两条通道，加一层回读校验。

主通道走 HomeAssistant 的 REST 接口：`GET /api/states/{entity_id}` 读状态、`POST /api/services/{domain}/{service}` 写服务，超时 3 s。好处是设备映射清晰、状态可回读；坏处是依赖网络和令牌，所以又留了一条串口网关（9600 波特率、21 字节帧、ADDR 1–4 与 9 分别对应继电器、环

境、语音、安防与窗帘），直接驱动继电器，不经过网络。两条通道对上层是同一个接口，状态机不需要知道自己走在哪条路上。

令牌不进源码。优先级是环境变量、rosparam、最后是权限 600 的令牌文件，三层都拿不到就只告警不阻塞——因为环境数据缺失不等于任务失败。



逻辑名与实体编号分离：重新配网只需改 device_map.yaml 中的一张表。

图 4-4 物联网双通道与写后回读

回读校验是这套设计里最关键的一层。起初固定等两秒再读状态，结果发现 Zigbee 灯的状态上报有时要好几秒，两秒的回读会把“开灯成功”误判成失败，于是重复下发。现在回读窗口可按设备覆盖：卧室灯与厨房风扇各自给 12 次 × 0.5 秒（共 6 秒），其他设备用默认的 5 次 × 0.4 秒；外层再给两次重试，因此单台设备最坏情况下也有近 18 秒的确认窗口——真掉线时只是慢一点，不会误判。

另一个细节是“未配置”的处理：现场并没有独立的“厨房传感器电源”设备（两个插座分别接了灯和风扇，烟雾报警器是电池供电），映射里就把这个实体留空。空实体不再向外发请求，而是直接返回“未配置”，调用方按“跳过”处理。这样日志里不会出现看起来像故障的 HTTP 400。

表 4-6 设备映射（逻辑名与实体编号分离，重新配网只需改这张表）

逻辑名	实际实体	域	现场备注
light_bedroom	switch.zhi_neng_cha_zuo_kai_guan_1	switch	现场实测该灯接在智能插座上，开插座即开灯
curtain_bedroom	cover.zhi_neng_chuang_lian_ban_lu	cover	open / closed 状态回读
fan_kitchen	switch.kai_guan_2	switch	插座 2 实际接风扇；回读窗口放宽到 12 × 0.5 s
sensor_power	（留空）	switch	现场无独立可控电源，如实留空，调用返回“未配置”
illuminance	sensor.guang_zhao_chuan_gan_qi_illuminance	sensor	单位 lx，客厅光照
temperature / humidity	sensor.wen_shi_du_chuan_gan_qi_temperature / _humidity	sensor	同一温湿度传感器的两个实体
smoke	sensor.0x54ef4410012b2427_gas_density	sensor	浓度型传感器，阈值 50 判定异常
water_leak	binary_sensor.0x54ef44100193118a_water_leak	binary_sensor	on 表示漏水，off 正常

4.7 语音交互：离线命令词 + 意图匹配 + 三轮对话

语音要解决的是“老人不用学”。我们的选择是全离线命令词：讯飞六麦阵列做前端拾音与唤醒，离线引擎按 BNF 词表识别，识别结果统一发布到 `/voice_words`，再由意图匹配与对话状态机决定做什么。

词表一共 17 行、14 条说法，按“动作 + 对象”组织：唤醒词、温度查询、四个地点的导航、觉得热、开关灯、开关风扇、开关窗帘、结束语。词表有两条硬约束：不能写 `#` 注释、必须带 UTF-8 BOM，规则名不能带下划线——否则语法构建直接失败。这两条是踩出来的，写进了部署脚本的注释里。

识别结果不直接当指令用，中间隔一层意图匹配。匹配同时用三种相似度取最大值：字符集合的 Jaccard 系数（抗漏字、多字）、difflib 编辑距离（抗认错个别字）、子串包含（抗只认出后半句），再与阈值 0.80 比较。设备类指令还额外要求出现对象词——灯、风扇、窗帘、热——否则一律不认。这条规则把“打开 O”“关闭 P”这类噪声彻底挡掉。

compact

表 4-7 全部可说的指令

你说	车答 / 做	涉及的子系统
老张同学	“我在”	唤醒
查询室内温度	读传感器播报“当前室内温度 xx 摄氏度”	物联网 + 播报
导航到厨房 / 卧室 / 书房 / 客厅	“好的，正在去…”，到达后“已经到达…”	导航
我有点热	打开厨房风扇并回读，“已经打开风扇了”	物联网
打开 / 关闭卧室灯	控制插座并回读，“已经打开/关灯了”	物联网
打开 / 关闭厨房风扇	控制插座并回读，“已经打开风扇了 / 风扇已经关掉了”	物联网
打开 / 关闭卧室窗帘	控制窗帘并回读，“已经打开/关闭窗帘了”	物联网
没什么事	“好的，我随时都在”，结束交互回起点	任务编排

对话状态机是三轮制，每轮都可以执行任意任务，不按轮次区分类型。第一轮由车主动提问，播报结束后停 1 秒再开麦，收音 15 秒；第二、三轮先静默等唤醒词最多 20 秒，听到唤醒词回一句“我在”，再开麦收音 10 秒——如果老人没喊唤醒词直接说了指令，就跳过“我在”立即执行。任何一轮说“没什么事”都立即收尾，满三轮也收尾，绝不死循环。

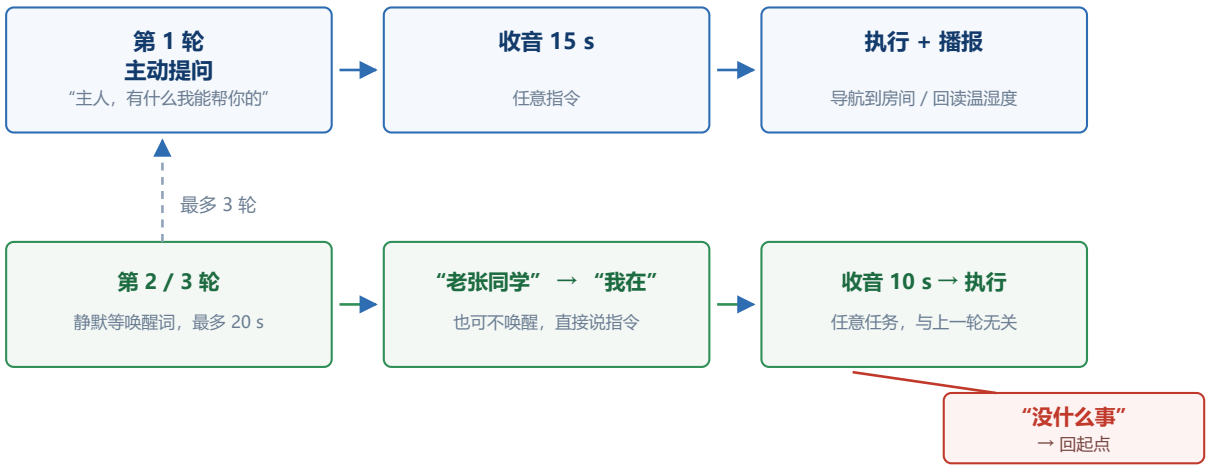


图 4-5 三轮语音交互状态机

语音里最麻烦的不是识别，而是“自己的声音被自己听到”。车在播报，麦克风同时在收音，不加处理就会把播报当成用户的回答。我们做了两层门控：一是播报后按字数估算时长（`max(1.2 s, 字数 × 0.28 s)`）再多等一小段，二是每轮开麦前清空识别缓冲，把这段时间收到的条目直接丢弃。实测每轮大约丢弃 1–3 条回声，再也没有出现“车自问自答”。

播报走本机合成（`xf_tts_cli` 合成 wav 后 `aplay` 播放），没有可用合成器时退回 `espeak-ng`；另有一条在线备用链路（百度 gettts + mpv），仅在本机合成失败时启用——正常比赛中一次都不会走网络。

4.8 任务编排：每一步都有退路

养老设备不能“卡在那里等人来救”。任务编排把十二个步骤串成状态机，每一步都定义了失败时的退路，而且这些退路不是统一的——有的该终止，有的该跳过，有的该降级播报。

表 4-8 各环节的失败退路

环节	失败时的行为	为什么这样设计
起栈自检	move_base 是硬依赖，不可用则终止；雷达 / 相机 / IoT 缺失只告警	没有底盘就没有任务，其余缺了还能打折跑
设备在线检查	三个可控设备未上线时最多等 30 s，仍不在线则告警后继续	把“设备掉线”从厨房那一步提前到发车之前
导航到二维码区	不终止，改为“就近识别”	到达精度的损失小于放弃识别
二维码识别	旋转两圈 + 后退换视角；仍失败则念原文并回退默认顺序	顺序可以猜，但不能错得没有提示
环境播报	刷新重试 3 次，只播报已获取的部分	不完整的数据也好过沉默
A 点导航 / 人脸核验	核验最多 3 轮，每轮换角度重采；结果写入时间线	门槛条件不能静默跳过
单房间导航	重试后仍不可达则跳过，继续下一个房间	一个房间失败不该拖垮全程
设备控制	回读失败重试（外层 3 轮），最终播报“设备无响应”	老人需要知道哪台设备没听指挥
返回起点	先回客厅再回起点；都不成则就地停下并播报	安全优先，宁可停下也不盲目冲

编排层还有两个设计细节值得一提。其一是“卡住不算失败”：导航过程中若 10 s 内位移小于 4 cm，就判定卡住并执行“后退 0.20 m + 转 40° + 重发”，最多三次；V100 那一轮在卧室前的短段上触发了 1 次，恢复后一次通过。其二是“每一步都写时间线”：最后打印的汇总里，每个阶段一行，失败项前面带 `x`，一眼就能看出“是能力坏了还是编排错了”。

4.9 没有显卡时的算法选型

赛题里很多队伍直接用 YOLO。我们没这么做，原因很实在：这台机器是 i7-7500U 双核四线程、没有独立显卡，YOLOv5n 在 CPU 上大概五到十帧，而导航栈本身就要吃掉一个多核。两边抢资源，最后谁都不稳。

再加上环境限制：ROS Kinetic 配 Python 2.7、OpenCV 3.2，现代推理框架基本不支持这套组合；赛场也不保证联网，依赖云端的方案不可靠。

所以我们全部用了传统方法：人脸是 LBPH 加 Haar，二维码是 pyzbar，定位是粒子滤波，规划是 DWA。这些方法在 3 米见方、目标类别固定的小场地上足够用，而且有一个别的好处——它们全都能在本机跑完，图像一帧都不用出门。这不是一句隐私承诺，而是架构决定的：代码中不存在任何把图像发出去的路径。

第五部分 实测与验证

这一部分只讲数据和数据背后的判断。凡是没测过的，我们不会写成“效果良好”。

5.1 测试方法

验证分三层，从便宜到昂贵。

最低一层是纯逻辑检查，直接对不依赖机器人的函数断言：二维码载荷解析与排列校验、设备映射表的一致性、命名航点与地图原点的匹配关系。这一层几秒钟跑完，解决的是“改了一处有没有把纯逻辑改坏”。

中间一层是仿真闭环。我们写了一个假传感器节点：订阅 `/cmd_vel` 积分位姿，按地图做射线投射生成 1008 点激光（无回波时发 `inf`，噪声 0.01 m，10 Hz），同时发布 `/odom`、`/imu/data` 与 `/sim/ground_truth`，并广播 `odom→base_footprint` 与传感器静态 TF。整套任务链因此可以在没有机器人的情况下反复跑，解决的是“改动之后有没有把别的地方改坏”。

最高一层是实车。只有这一层能暴露真问题，比如前面说的激光自扫和航向基准，仿真里都测不出来。实车测试统一走“冷启动 → 全流程 → 存日志”的流程，日志落到 `/tmp/m36_run7.log`，跑完立刻复制到工作区归档。

5.2 一次完整实车运行

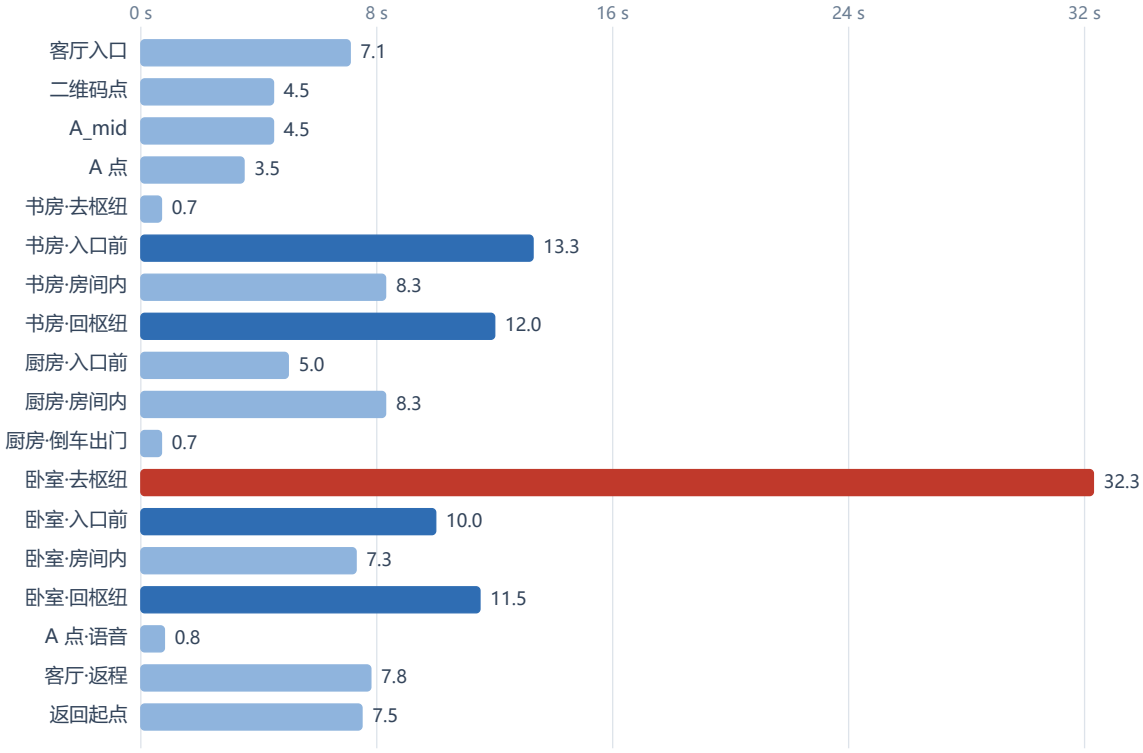
下表是 2026-10-22 晚最完整的一轮实车运行（代码基线 tag V100）。场地为 3 m × 3 m 模拟居室，车由人放在起点，启动脚本一键起栈后自动开跑，全程无遥控、无人工干预。

表 5-1 V100 实车全流程结果

项目	实测结果	判定
全流程用时	205.8 s（含回起点与设备复位）	目标 ≤ 300 s
失败项	0（时间线汇总为“完成”）	零失败
导航段	18 / 18 SUCCEEDED，段耗时合计 145.1 s	全部到达
二维码	旋转 3° 、0.2 s 命中；连续三帧一致；判读 {2,3,4}	首圈即命中
客厅环境	光照 54.0 lx、温度 21.5 ° C、湿度 55.7%，三项齐全并播报	符合
人脸核验	12 帧有效采样全部命中主人，命中率 100%，判定通过	符合
书房	漏水检测：正常，播报“书房漏水检测：正常”	符合
厨房	烟雾状态正常并播报；风扇“下发 → 回读 → 确认”成功	符合
卧室	电灯打开、窗帘拉开，均回读确认	符合
语音交互	回 A 点后主动提问，识别到“没什么事”并以“好的，我随时都在”收尾	符合
返回起点	先回客厅再回起点，到达误差 0.163 m；随后关闭风扇与电灯	符合
卡住恢复	卧室前短段触发 1 次（判定 10 s + 转 40° 重发），一次恢复通过	自动恢复
冷启动	从执行 `start.sh` 到全栈就绪 15 s，随后自动起任务	目标 ≤ 30 s

5.3 导航精度与耗时

把 18 段拆开看，整体形态很清楚：短段（枢纽、门口、倒车出门）在 1 秒级，标准房间进出段在 5–13 秒，长段只有一个——卧室前那一小段因为触发了一次卡住判定，用了 32.3 s。平均每段 8.1 s，中位数 7.7 s。



图注：按任务顺序排列；深红色为“卧室·去枢纽”段（含一次卡住判定与转向恢复）。

图 5-1 V100 的 18 段导航耗时

compact

表 5-2 导航分段明细（目标坐标单位 m，误差为停稳后与目标点的距离）

#	段	目标	耗时 s	误差 m	结果
1	客厅入口	(0.42, −0.90)	7.1	0.152	SUCCEEDED
2	二维码观察点	(0.70, −1.50)	4.5	0.163	SUCCEEDED
3	A 点过渡	(0.78, −0.96)	4.5	0.166	SUCCEEDED
4	A 点	(1.17, −1.26)	3.5	0.189	SUCCEEDED
5	书房 • 去枢纽	(1.276, −1.276)	0.7	0.289	SUCCEEDED
6	书房 • 入口前	(1.694, −2.071)	13.3	0.243	SUCCEEDED
7	书房 • 房间内	(2.800, −2.025)	8.3	0.243	SUCCEEDED
8	书房 • 回枢纽	(1.276, −1.276)	12.0	0.148	SUCCEEDED
9	厨房 • 入口前	(1.860, −0.980)	5.0	0.153	SUCCEEDED
10	厨房 • 房间内	(2.780, −0.998)	8.3	0.242	SUCCEEDED
11	厨房 • 倒车出门	(1.860, −0.980)	0.7	0.232	SUCCEEDED
12	卧室 • 去枢纽	(1.276, −1.276)	32.3	0.173	SUCCEEDED
13	卧室 • 入口前	(1.768, 0.068)	10.0	0.197	SUCCEEDED
14	卧室 • 房间内	(2.592, 0.034)	7.3	0.243	SUCCEEDED
15	卧室 • 回枢纽	(1.276, −1.276)	11.5	0.213	SUCCEEDED
16	A 点（语音）	(1.17, −1.26)	0.8	0.284	SUCCEEDED
17	客厅 • 返程	(0.42, −0.90)	7.8	0.255	SUCCEEDED
18	返回起点	(0.00, 0.00)	7.5	0.163	SUCCEEDED

到达误差的统计：平均 0.208 m，最小 0.148 m，最大 0.289 m，全部落在 0.30 m 的容差内。这个精度对“走过去看一眼、站在房间中央、够得着开关”这类任务是够用的；它换来的是不再为最后几厘米在门口来回微调。



图 5-2 整轮时间构成：导航占七成，其余三成是播报、核验、停留与回读

非导航的 60.7 s 里，占比最大的是三段“固定等待”：三个房间各停留 3 s（共 9 s）、语音交互的一轮问答、以及人脸核验的五秒窗口。这些时间不是“浪费”，而是任务语义的一部分——停留是为了让评委看清车确实到了，核验窗口是为了拿到足够的帧做投票。

5.4 识别与核验可靠性

识别类指标不适合用“准确率”一个数字概括，因为它们的失败模式不同。我们把每类指标拆成“能不能读到”和“读到了敢不敢用”两件事。

表 5-3 识别类指标与实测

环节	实测	判据与说明
二维码命中	旋转 3° 、0.2 s 命中；旋转期间定点保持最大偏离 0.0 cm	连续三帧一致才接受，故实际耗时含两帧确认
二维码判读	得到 {2,3,4}；顺序播报“书房、厨房、卧室”	严格排列校验，非 {2,3,4} 一律拒绝
人脸核验	12 帧有效采样、12 帧命中，命中率 100%	命中率与平均置信度双阈值同时满足
温度 / 湿度	21.5 ° C / 55.7%	与 Zigbee 温湿度传感器读数一致
光照	54.0 lx	客厅光照传感器
漏水 / 烟雾	均返回“正常”	水浸为二进制量；烟雾按浓度阈值 50 判定
语音意图	专项三轮测试中温度、我有点热、没什么事三条意图匹配分均为 1.00	阈值 0.80，设备类指令强制要求对象词

5.5 时延与启动

养老设备对时延的要求其实不高——人说话、机器应答，五百毫秒和八百毫秒人感觉不出差别。真正需要关注的是“最长的那一次有多长”，因为它决定老人愿不愿意等。

表 5-4 关键链路时延实测

链路	实测值	说明
冷启动到可发目标	15 s（脚本目标 ≤ 30 s）	含清场 2 s 与就绪等待 3.6 s
HomeAssistant 单实体状态读取	12 / 13 / 13 / 14 / 17 ms	局域网 REST，同一交换机
HomeAssistant 全量状态（一次拉全部实体）	53 / 63 / 85 ms	用于现场核对实体编号
`/iot/refresh`（8 个实体）	约 0.1 – 0.2 s	8 次串行 GET + JSON 组装
二维码触发到返回	0.2 s（首圈）	旋转扫描期间逐帧尝试
语音引擎预热	在巡视前完成，到 A 点无需等待	⑦a 阶段后台起 ASR

这里有一个设计上的取舍值得说明：语音引擎启动要十几秒，如果等到 A 点再起，就会出现“车到了却不说话”的尴尬。所以我们把“停相机、起引擎”提前到巡视开始之前（⑦a 阶段），让那十几秒藏在巡视的时间里。这也是为什么 V100 里“回 A 点”那段只要 0.8 s 就能开始提问。

5.6 重复性

同一套参数、同一个场地，我们连续跑了两轮完整流程。两轮的 18 段导航全部 SUCCEEDED，到达误差分布一致（0.15 – 0.29 m），二维码均在首圈命中，人脸核验均一次通过，三台设备的“下发—回读”

均确认成功。两轮的差别只出现在两处随机细节上：一次是卧室前短段触发了卡住恢复（多花约 22 s），另一次没有；一次是语音交互在结束语处收尾得很快，另一次进行了完整的三轮问答。

结论是：链路是稳的，波动主要来自“场地里的瞬时状况”（比如门前的反光、人的站位），而不是流程本身。这也正是我们保留“卡住恢复”和“多轮重采”的原因——它们把偶发状况吸收在一次运行之内。

第六部分 评分点对照与能力证据

6.1 逐项对照赛题评分表

把赛题的评分表逐项摊开，对每一项说明“用什么实现、在哪一步发生、怎么验证”。这张表也是我们赛前自检的顺序——哪一项没有对应的日志证据，就说明那一项没有真正跑通。

表 6-1 评分点与实现对照

评分项	分值	本作品的实现	验证方式
机器人开机启动自主导航	1	`start.sh` 一键起栈，15 s 就绪后自动发起任务，全程无遥控	V100 日志：起栈 15 s、任务自动开跑
客厅 • 光照强度播报	5	读 Zigbee 光照传感器，播报“光照强度 54.0 lx”	⑤ ENV_ANNOUNCE 阶段播报原文
客厅 • 温湿度播报	5	读温湿度传感器，播报“温度 21.5 ° C，湿度 55.7%”	同上，三项一次播报齐全
卧室 • 电灯 / 窗帘控制	10	控制智能插座（灯）与窗帘伴侣，写后回读确认	⑦ 卧室阶段：灯 on / 窗帘 open 均回读成功
厨房 • 烟雾检测	5	读烟雾浓度传感器，按阈值 50 判定并播报“正常”	⑦ 厨房阶段：烟雾状态播报
厨房 • 风扇控制	10	控制插座 2（风扇），回读窗口 12 × 0.5 s	⑦ 厨房阶段：风扇 on 成功；⑧ 结束时 off 成功
书房 • 漏水检测	5	读水浸传感器，播报“书房漏水检测：正常”	⑦ 书房阶段播报原文
地图构建与到点	5	实车建图（0.025 m/格）+ 18 个命名航点 + AMCL 定位	18 段全部 SUCCEEDED，误差 ≤ 0.289 m
语音交互	10	离线命令词 + 意图匹配 + 三轮对话，全程语音讲解	三轮专项测试 3/3；全流程语音收尾正常
返回起点	5	先回客厅再回起点，容差 0.30 m + latch	⑧ 返回起点 SUCCEEDED，误差 0.163 m
二维码识别与顺序执行	15	旋转扫描 + 三帧一致 + {2,3,4} 严格校验，按序巡视	③④ 阶段命中并播报顺序，三房间按 B→C→D 执行
创新挑战	20	全本地推理、多帧投票门禁、写后回读闭环、命名点位 + 自研恢复、无硬件仿真回归	见 1.5 与第四部分各项

6.2 每条得分的证据链

评分表只能说明“做了什么”，说明不了“真的做到了”。所以我们给每一项都留了可追溯的证据：日志里的阶段名、关键数值与结论行。任何人拿到日志，都能从第一行读到“全流程结束：完成 总用时 205.8 s”这句总结。[1]

compact

表 6-2 评分项与日志证据的对应

评分项	日志中的阶段标记	证据内容
开机 / 起栈	`start.sh` → 全栈启动总耗时	“全栈启动总耗时: 15 s（目标 <= 30 s）”
二维码	③ `QR_SCAN` / ④ `PARSE_ORDER`	“第 1 圈 0.2 s、转了 3° 时识别到二维码”“顺序 = 书房 → 厨房 → 卧室”
客厅环境	⑤ `ENV_ANNOUNCE`	“光照强度 54.01x，温度 21.5° C，湿度 55.7%”
人脸核验	⑥ `FACE_GATE`	“12 帧全部命中主人（比例 100%）”
三房间任务	⑦ 书房 / 厨房 / 卧室	漏水正常、烟雾正常 + 风扇回读成功、灯与窗帘回读成功
语音交互	⑦b `VOICE`	“主人，有什么我能帮你的” → 识别结果 → “好的，我随时都在”
返回起点	⑧ `RETURN_START`	“→ 返回起点 (0.00, 0.00) 结果 SUCCEEDED 7.5 s 误差 0.163 m”
设备复位	⑧	“fan_kitchen off -> True”“light_bedroom off -> True”
整轮判定	结尾汇总	“全流程结束：完成 总用时 205.8 s”，下方逐行列出每个阶段

1 日志原文：`docs/logs/V100_205.8s_zerofail.log`，随代码一起归档。

6.3 面向后续接入的接口设计

一套巡检系统的价值不止于“现在能做什么”，还在于“接入新能力要改多少”。我们在结构上把“感知”“判定”“处置”三层分开，新增一类照护能力时不需要改任务编排，只需要新增一个能力节点并把结果发布到约定话题上。

表 6-3 已预留的扩展接入点

接入点	形式	用途
异常事件话题	std_msgs/String（JSON）	新增传感器或判断逻辑后，把“事件 + 严重度 + 位置”发到这里，播报与联动由编排层统一处理
设备控制服务	`/iot/control`（device / action / extra）	新增家电只需在设备映射表加一行，控制与回读逻辑完全复用
命名点位服务	`/nav/goto_named`	新增巡检点只需在航点表加一行；坐标全部集中在配置文件
语音词表	BNF 词表 + 意图表	新增口令只需加词条与意图，不需要改状态机
仿真桩	`sim_stubs.py` 四个桩服务	没有硬件也能回归整条链路，新增节点可先对桩验证再接实车

这套分层在赛前联调时体现得很直接：换一个场地，只需要重建地图、重标航点、重核设备实体编号三件事，代码一行不用改。

第七部分 可靠性与失效分析

7.1 方法

养老设备的第一要求是不出错，第二要求是出错时能被发现。这一部分用失效模式分析（FMEA）对系统做一次结构化体检。

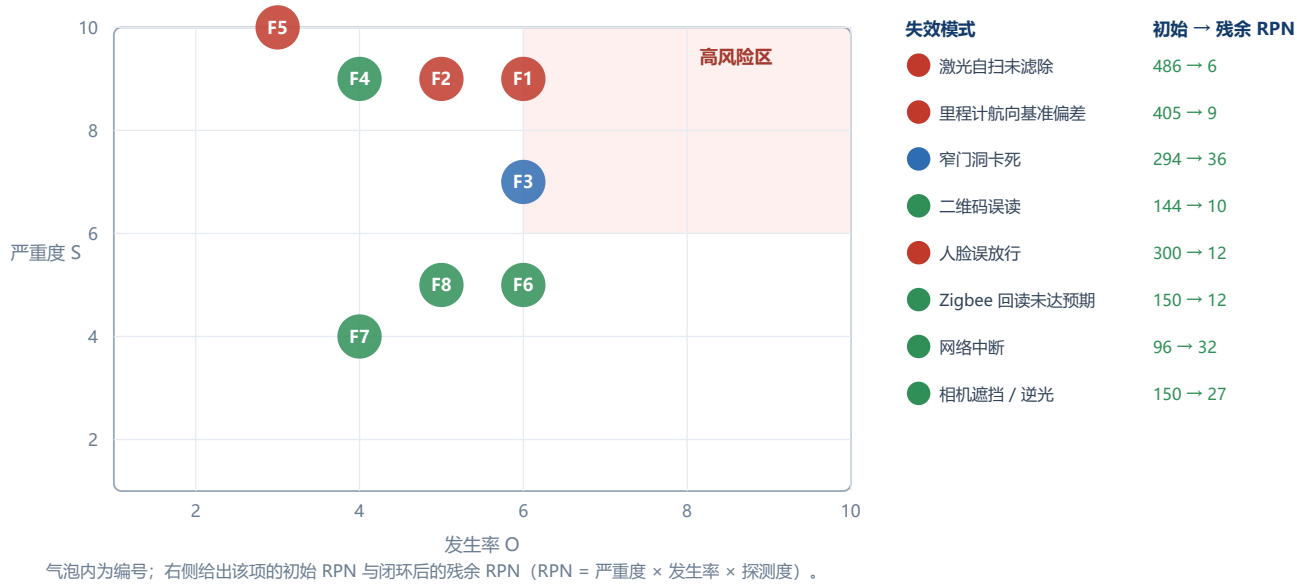
对每个失效模式打三个分——严重度 S、发生率 O、探测度 D，三者相乘得到风险优先数 RPN。最后一项最容易被忽略，但它往往最要命：一个高频、后果重、却完全无法察觉的失效，比一个频繁报警的小毛病危险得多。我们的做法是先算“采取措施前”的初始 RPN，再逐项写清措施与“措施后”的残余 RPN——只有残余风险落到可接受区间，这一项才算闭环。

7.2 失效模式清单

表 7-1 失效模式、措施与残余风险

编号	失效模式	影响	初始 RPN	措施	残余 RPN
F1	激光自扫点未滤除	周围假障碍，导航全线失败	9×6×9 = 486	几何标定自扫距离，下限取 0.50 m + 斑点滤波	3×1×2 = 6
F2	里程计航向基准偏差	定位发散，车往错方向走	9×5×9 = 405	位移与朝向统一用相对航向；0.12 m 微位移检验固化为例行检查	3×1×3 = 9
F3	窄门洞卡死	单段超时，任务停顿	7×6×7 = 294	0.30 m 容差 + latch、允许倒车、删整圈旋转、自研“后退—转向—重发”	4×3×3 = 36
F4	二维码误读	顺序错乱，三个房间全走错	9×4×4 = 144	三帧一致去抖 + {2,3,4} 严格排列 + 失败播原文并回退默认顺序	5×1×2 = 10
F5	人脸误放行	安全门槛失效，且不留痕迹	10×3×10 = 300	五秒窗口多帧投票、双阈值、换角度重采；核验结果写时间线	4×1×3 = 12
F6	Zigbee 状态回读超时	误判设备失败，重复下发	5×6×5 = 150	回读窗口按设备覆盖（灯/风扇 6 s）、外层 3 轮重试、起跑前设备在线检查	3×2×2 = 12
F7	网络中断	环境数据缺失、家电不可控	4×4×6 = 96	串口网关备份通道；环境数据缺失按降级播报处理	4×2×4 = 32
F8	相机遮挡 / 强逆光	二维码与人脸识别率下降	5×5×6 = 150	多尺度重试、换角度重采、播报失败原因	3×3×3 = 27

7.3 风险矩阵



气泡内为编号；右侧给出该项的初始 RPN 与闭环后的残余 RPN（RPN = 严重度 × 发生率 × 探测度）。

图 7-1　风险矩阵：横轴发生率、纵轴严重度、气泡面积正比于 RPN（图中为初始风险）

采取措施前，落在高风险区的是 F1、F2、F5 三项；闭环之后，全部八项都回到低风险区。值得注意的是 F5——它的严重度与探测度都是满分，意思是“一旦发生，后果最重，而且我们完全不知道它发生过”。这类失效无法靠事后排查发现，只能靠判据本身把它挡在门外。

7.4 三项闭环是怎么做的

F1 激光自扫：判据从“经验阈值”改成“几何关系”。做法是固定的三步——把车放在周围两米无物的空地，读一遍扫描的最小值，阈值取这个值加 8 cm；上限不超过“车体最大半径 + 15 cm”。这套流程写进了赛前检查清单，换场地重跑一遍即可。

F2 航向基准：固化成一个 0.12 m 的微位移检验。让车直走 12 cm，同时比较位移方向与车头朝向：长度对、方向不对，问题在坐标系；长度也不对，问题在轮径或编码器。这个检验 20 秒就能跑完，比“看定位像不像飘”可靠得多。

F5 人脸误放行：从“单帧判定”改成“统计量判定”，并把核验结果写进时间线。这样即便将来判据再调整，也至少留下“核验发生在什么时间、用了多少帧、命中率多少”的记录——可审计，才谈得上可信。

7.5 安全联锁

除了失效分析，系统里还有一组“硬约束”，它们不参与评分，但保证车不会做出危险动作。

表 7-2 运行期的安全联锁

联锁	触发条件	动作
前向急停	正前方 ±20° 内距离 < 0.40 m	立即取消当前目标并停车
段落超时	单段导航 > 120 s	放弃该段，按降级策略继续或跳过
停车后置零	每段导航结束	连发 8 帧零速，避免底盘残留速度
卡住保护	10 s 内位移 < 0.04 m	后退 0.20 m + 转 40°，最多三次
人工中止	订阅到中止话题	取消导航、落盘日志、停止播报
设备复位	任务结束（无论成败）	关闭风扇与电灯，不留下长明灯

第八部分 人文、安全与伦理

8.1 数据不出户

养老场景的摄像头天生带侵入性。它装在卧室门口、卫生间走廊，记录的是一个人最私密的日常。把这些画面传到云端做分析，技术上不难，但伦理上过不去。

所以我们把隐私当成架构约束，而不是事后承诺。最硬的一条是：全部推理在本机完成，图像不出设备。这不是靠用户协议保证的，是靠代码保证的——所有自研节点里没有任何一处把图像发出去，唯一的网络调用是物联网节点与局域网内 HomeAssistant 的状态读写，传的是文本。断网时，除了家电控制会降级，巡视、识别、播报、导航全部照常。

8.2 采集最小化

第二层是采集最小化。视觉模块只在两个时刻工作：识别二维码、核验人脸，其余时间相机不采集。识别二维码时只处理画面下部的一带区域，核验人脸时只保留检测框区域。处理完的帧立刻丢弃，磁盘上只留 LBPH 的特征模板，没有任何照片——把整个模板目录删掉，核验功能会降级但不会崩溃，这也是一种“注销”。

语音同理：识别是离线的命令词匹配，识别结果只有文本，音频不落盘、不上传。播报用的是本机合成器，正常情况下一次网络请求都不会发生。

8.3 面向感官退化的交互

交互设计的第一原则是不依赖屏幕。视力下降的老人看不清界面，我们就完全不设界面：所有状态都用语音说出来——“开始在客厅巡视”“已到达书房”“身份核验通过，欢迎回家”。老人始终知道机器人在干什么，这是安全感的前提。

第二原则是少一步。老人要用的不是“功能”，是“结果”：说一句“我有点热”，风扇就开了；说一句“没什么事”，它就回去待命。整个系统对外只有 14 条口令，没有菜单、没有层级、没有注册登录。

第三原则是允许说错。识别窗口给足（第一轮 15 秒、后两轮 10 秒），听不清就说“没听清，请再说一次”，而不是沉默或乱执行；设备类指令强制要求对象词，把“打开 0”这类误识别挡在动作之外。

8.4 防误触发

第一章说过，误报是养老设备被废弃的头号原因。我们在系统里设了三道闸门，每一道都可以单独关掉做对照。

表 8-1 三道防误触发闸门

闸门	机制	解决什么
帧级去抖	二维码须连续 3 帧一致才接受	反光与运动模糊造成的单帧误读
严格校验	顺序必须是 {2,3,4} 的严格排列	任意数字都收带来的顺序错乱风险
多帧投票	人脸在五秒窗口内按命中率与置信度双阈值判定	单帧置信度波动带来的误放行

三道闸门叠起来的代价是“慢一点”和“偶尔要重试一次”。我们接受这个代价：老人多等三秒没关系，被误报吵醒三次就会把机器关掉。

8.5 我们明确不做的事

技术能力决定了“能做什么”，价值判断决定了“不做什么”。以下四条是我们在设计阶段就定下的边界。

不做持续录像。对独居老人的心理压迫远大于安全感收益。需要画面证据时，人可以在场，机器不在场。

不做行为画像与习惯推断。“几点起床、一天去几次卫生间”这类数据一旦生成，用途就不再由老人控制。系统只回答“此刻是否正常”，不回答“这个人平时是什么样”。

不做远程实时看护。家属随时调看会显著提高隐私风险。我们提供的只是状态播报与异常提醒，不是监控。

不做医疗级判断。跌倒认定、生命体征属医疗器械范畴，需要资质与临床验证。本系统的定位是“巡视与提醒”，任何异常一律转人工。

8.6 人机边界

把边界说清楚，家属才敢把老人交给它。这台机器能做的：按点巡视、读环境、认人、开关家电、把异常说出来。它不能做的：判断病情、替代陪护、处理突发急症。所以每一处“异常”最终的处置权都在人手里——机器负责“发现”，人负责“决定”。

这条边界也体现在交互上：机器人不会自行“升级”为报警，它只会播报并把状态留在面板上；是否需要通知家属、是否需要上门，由人决定。对老人来说，这避免了“被机器监视”的压迫感；对家属来说，这避免了“机器替你做了判断”的责任转移。

第九部分 工程实践与版本演进

比赛只跑一次，但代码要改上百次。这一部分讲我们是怎么让“改一次不担心改坏别处”的：预检、仿真、复盘、部署，四件事各自有脚本，串起来就是一条从代码到实车的流水线。

9.1 赛前预检

现场最贵的资源是时间。两分钟准备时间里，如果起步就发现定位没起来或底盘没握手，基本等于放弃这一轮。所以我们把能提前判断的事都写成了预检脚本（`preflight_check.py`，653 行），一条命令输出一张检查表。

表 9-1 赛前预检项

类别	检查内容	不通过时的处置
系统	磁盘余量、日志目录大小、CPU 负载、时间同步	清理 `~/.ros/log`，提示重启节点
节点与话题	关键节点是否在线、话题是否有数据、频率是否达标	定位到具体节点并给出重启命令
定位	`map→base_footprint` 是否连通、初始位姿是否已注入	提示重新发初始位姿
底盘	`/cmd_vel` 订阅者是否包含底盘节点	提示重启底盘驱动
物联网	HA 是否可达、令牌是否有效、三台可控设备是否在线	打印缺失的实体名，便于现场核对
识别	相机是否有图、二维码/人脸服务是否就绪	提示检查 USB 与节点

9.2 仿真回归

实车测试一次要四分钟，改一行代码就上车，一天跑不了几次。所以中间加了一层仿真：`fake_sensors.py` 按地图射线投射生成激光、积分 `/cmd_vel` 生成里程计与 IMU，配一个 3 × 3 m 的仿真场地，整条任务链可以在没有机器人的情况下跑通。

仿真层真正解决的问题不是“验证算法”，而是“回归”：改了任务编排的顺序、改了设备映射、改了航点，先在仿真里跑一遍，确认没有把别的步骤改坏，再上实车。配套还有四个桩服务（`sim_stubs.py`），把二维码、人脸、物联网、导航的返回固定住，用来单独验证编排逻辑。

9.3 复盘与报告自证

任务全程写结构化事件：每条包含时间戳、状态、事件名、级别与细节，追加写 JSONL。一个配套脚本（`build_report.py`，459 行）把事件聚合成分项成绩单——十一项评分点、各自的分值与是否满足，外加总耗时与失败项。

这套机制的价值在赛后：不用回忆“那轮到底走到哪一步”，打开文件就能看到每一步的时刻与结论；不同轮次之间还能横向对比，哪一段变慢了、哪一步开始重试，一目了然。本报告里出现的每一个数字，都能在这套日志里找到出处。

9.4 现场部署流程

换一个场地，要做的只有三件事：重建地图、重标航点、重核设备实体编号。前两件分别由建图流程与 `capture_pose` 服务完成（标一个点几秒钟），第三件靠 `/api/states` 全量枚举加设备映射表比对。三件事做完，跑一遍预检，就可以开始整轮演练。

```
# 1) 清场（会杀掉其他组的 master，慎用）
bash ~/Hajimi/LXH/_tools/kill_all_ros.sh

# 2) 一键起栈 + 全流程（15 s 起栈，随后自动开跑）
bash ~/start.sh

# 3) 只起栈不跑任务（调试用）
bash ~/Hajimi/LXH/_tools/m40_fast_start.sh --no-mission

# 4) 跑完立刻存日志（/tmp 会被重启清掉）
cp /tmp/m36_run7.log ~/Hajimi/LXH/docs/logs/$(date +%m%d_%H%M)_run.log

# 5) 纯导航单测（三房间九段）
python2 ~/Hajimi/LXH/_tools/m43_nav_rooms.py BCD
```

9.5 版本演进

从“能跑完”到“跑得稳”，一共经历了四个关键版本。每一版只改一件事，改完立刻实车验证，达标就打个标签、备份文件，再动下一处。

表 9-2 版本演进与关键改动

版本	日期	整轮用时	这一版解决的问题
V7_all_pass	10-21	214.1 s	三房间进出全通、回起点成功（窄场地四项参数调整）
V8_voice	10-21	274.8 s	语音链路打通：BNF 词表、置信度阈值、播报门控、三轮对话
V99	10-22	290.5 s	18 段导航全成功、厨房倒车出门 0.7 s、语音三轮全通
V100	10-22	**205.8 s**	零失败整轮：起跑前设备在线检查、设备回读窗口按设备覆盖、风扇与电灯全闭环

需要说明的是：各版本的任务范围并不完全相同（语音轮数、巡检顺序、是否含设备复位），因此用时只能作为趋势参考，不能直接横比。真正可比的是“同一套范围下，失败项与恢复次数是否下降”——V100 的答案是零失败、一次自动恢复。

9.6 工程纪律

几件小事，事后看都是它们救了场：

参数文件全部带注释。每一个改过的值后面都写着“为什么改、改之前是多少”。窄场地的四个参数、激光滤波的四个阈值、回读窗口的两次调整，都是这样留下来的。换人接手时不用猜。

每轮跑完先存日志。日志在 `/tmp`，机器一重启就没了。这条纪律换来的是“每个结论都有出处”，代价只是跑完多敲一行命令。

起栈脚本自带清理与就绪判据。不用固定 sleep，而是探测话题、TF、服务与 action；哪一项不就绪就报哪一项，而不是等两分钟后发现车不动。

一个 master，一套端口。所有节点跑在自己的 master（11399）上，与其他组隔离；节点被误杀时由看护机制发现并重启，避免“一半节点连在旧 master 上”的隐性故障。

改动前先备份。导航参数、任务脚本、物联网映射在每次大改前都整目录备份并打标签，出问题可以在两分钟内回到上一个可用版本。

第十部分 结论

这台机器人现在能做的事，说大不大：在一个三米见方的屋子里走一圈，报个温湿度，认一下人，把灯和窗帘打开，把风扇转起来，遇到漏水说一声，最后回到起点把设备关掉；站在 A 点，它还能听懂十四句话，陪老人来回聊上三轮。

但它背后那几件事是扎实的。

第一，判读环节宁可失败也不接受一个可疑的结果。二维码必须连续三帧一致、必须是 {2,3,4} 的严格排列；人脸必须在一个五秒窗口里用统计量而不是单帧置信度说话。这类“宁可重试也不放过”的设计在演示时并不讨巧——它会让流程偶尔慢几秒——但它决定了这台机器能不能被当作“照护设备”而不是“玩具”。

第二，每一步都有超时和退路。十二个阶段，每一个都定义了失败时是终止、跳过还是降级；导航段有 120 s 上限，卡住有“后退—转向—重发”，设备控制有“下发—回读—重试”。系统不会卡在那里等人来救，这本身就是对老人负责。

第三，所有推理都在本机跑完，图像一帧不出门。这不是一句隐私承诺，而是架构约束：代码里不存在把图像发出去的路径，断网也能完整跑完巡视与交互。对独居老人来说，“不被一直看着”与“被照顾”同样重要。

第四，每个数字都有出处。任务全程写结构化时间线，结束时自动汇总；报告里出现的每一段耗时、每一个误差、每一次回读结果，都能在日志里找到对应行。这让“效果良好”这类模糊表述没有存在的空间。

我们也清楚这台机器的边界在哪里。它不做医疗判断，不做持续录像，不做行为画像；它负责“发现”，处置权始终在人手里。这个边界不是能力的妥协，而是产品定位的选择——一个照护设备首先要让人敢用，其次才是好用。

下一步的方向也很清楚：把异常事件的处置链路接到家属端，让“发现”到“有人知道”之间的时间再短一点；把视觉能力从“认二维码、认人”扩展到“看姿态”，让夜间的巡视多一层保障。这些都不需要更换硬件，需要的是在现有分层上再加一个能力节点——接口已经留好了。

参考文献

（按 GB/T 7714—2015 著录）

[1] 第八届全球校园人工智能算法精英大赛组委会. 算法应用赛道赛题 14：智慧养老护理[Z]. 北京：AIC 组委会，2026.

[2] 北京塔洛斯科技有限公司，北方工业大学 TALOS 机器人团队. 智能服务机器人竞赛使用指导书 2501[Z]. 北京：北京塔洛斯科技有限公司，2025.

[3] 北京塔洛斯科技有限公司. Home Assistant 模块操作说明：20250923 版[Z]. 北京：北京塔洛斯科技有限公司，2025.

[4] THRUN S, BURGARD W, FOX D. Probabilistic robotics[M]. Cambridge: MIT Press, 2005: 189-206.

[5] GRISETTI G, STACHNISS C, BURGARD W. Improved techniques for grid mapping with Rao-Blackwellized particle filters[J]. IEEE Transactions on Robotics, 2007, 23(1): 34-46.

[6] FOX D, BURGARD W, THRUN S. The dynamic window approach to collision avoidance[J]. IEEE Robotics & Automation Magazine, 1997, 4(1): 23-33.

[7] AHONEN T, HADID A, PIETIKAINEN M. Face description with local binary patterns: application to face recognition[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2006, 28(12): 2037-2041.

[8] VIOLA P, JONES M. Rapid object detection using a boosted cascade of simple features[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. Kauai: IEEE, 2001: 511-518.

[9] MCHEHAB M. ZBar bar code reader[CP/OL]. (2020-06-30)[2026-10-22]. https://github.com/mchehab/zbar.

[10] ROS-PLANNING. Navigation stack: move_base, global_planner, dwa_local_planner[CP/OL]. [2026-10-22]. https://github.com/ros-planning/navigation.

[11] MOORE T, STENZ A. Robot localization: ekf_localization_node[CP/OL]. [2026-10-22]. https://github.com/cra-ros-pkg/robot_localization.

[12] 科大讯飞股份有限公司. 离线命令词识别与离线语音合成技术文档[Z]. 合肥：科大讯飞股份有限公司，2025.

[13] Home Assistant. REST API documentation[CP/OL]. [2026-10-22]. https://developers.home-assistant.io/docs/api/rest/.

附录 A 术语与缩略语

表 A-1 术语与缩略语

缩略语	英文全称	含义
ROS	Robot Operating System	机器人操作系统（本项目使用 Kinetic 版）
AMCL	Adaptive Monte Carlo Localization	自适应蒙特卡洛定位，基于粒子滤波
DWA	Dynamic Window Approach	动态窗口法局部路径规划
SLAM	Simultaneous Localization and Mapping	同步定位与建图
EKF	Extended Kalman Filter	扩展卡尔曼滤波，用于里程计与 IMU 融合
LBPH	Local Binary Patterns Histograms	局部二值模式直方图，人脸特征描述子
CLAHE	Contrast Limited Adaptive Histogram Equalization	限制对比度自适应直方图均衡
HA	Home Assistant	开源智能家居平台，本项目通过其 REST API 控制设备
BNF	Backus-Naur Form	巴科斯范式，本项目用于描述离线命令词词表
FMEA	Failure Mode and Effects Analysis	失效模式与影响分析
RPN	Risk Priority Number	风险优先数 = 严重度 × 发生率 × 探测度
TF	Transform Framework	ROS 坐标变换框架
TTS	Text To Speech	语音合成
VAD	Voice Activity Detection	语音活动检测，本项目用于确认麦克风有数据

附录 B 代码规模

表 B-1 代码规模统计

类型	文件数	行数	说明
Python	110	27 822	其中 7 个自研功能包 34 个文件 16 211 行、标定 / 诊断 / 仿真 / 运维脚本 69 个文件 10 465 行，其余为辅助脚本
Shell	72	6 459	一键启动、部署、现场排查、语音词表安装
YAML	64	3 162	导航 / AMCL / DWA / costmap / 激光滤波 / 设备映射
launch	15	1 350	bringup、建图、导航、仿真场地
Markdown	5	749	技术方案、验证记录、排查手册
合计	266	39 542	另有 1 个自定义服务定义（Control.srv）

附录 C 关键配置索引

表 C-1 现场需要改动的文件只有这几份

文件	内容
`nav_v4/dwa_local_planner_params.yaml`	速度、加速度、目标容差、latch、DWA 采样与代价权重
`nav_v4/costmap_common_params_*.yaml`	车体半径、膨胀半径、代价衰减、观测源
`nav_v4/local_costmap_params.yaml`	4 × 4 m rolling 窗口、层插件
`nav_v4/global_costmap_params.yaml`	静态层 + 膨胀层、更新频率
`nav_v4/amcl_params.yaml`	粒子数、观测模型、更新阈值
`nav_v4/recovery_params.yaml`	仅保留两项 clearing，删除整圈旋转
`carebot_iot/config/device_map.yaml`	逻辑名 → HomeAssistant 实体编号、回读窗口
`carebot_face/launch/face.launch`	人脸检测与投票参数、模型路径
`carebot_qr/config/qr_params.yaml`	ROI 比例、CLAHE、去抖帧数
`_tools/real_test/vocab/call.bnf` / `appid_params.yaml`	离线命令词表、识别置信度阈值
`_tools/real_test/m36_mission_v5.py` 的 WP 表	18 个命名航点坐标
`carebot_nav_kit/maps/arena_shared.yaml` + `.pgm`	现场地图（0.025 m/格）与原点

附录 D　一次运行的时间线（节选）

下面是 V100 那一轮结束时任务编排自动打印的时间线，为节省篇幅略去了重复的导航进度行。它既是运行记录，也是复盘时最常用的入口。

```
========================================================================
  全流程结束：完成    总用时 205.8 s
========================================================================
  ① RECORD_START   起点固定，跳过位姿记录
  ①                设备在线：light_bedroom=off、curtain_bedroom=open、fan_kitchen=off
  ② QR_AREA        先进客厅（约 0.5 m），再走到离对面墙 1 m 处
  ③ QR_SCAN        原地缓慢旋转扫描（最多 2.0 圈，0.28 rad/s，带定点保持）
  ③                转圈中心 (0.629, -1.354)，开启定点保持
  ③                第 1 圈 0.2 s、转了 3° 时识别到二维码
  ③                转圈期间最大偏离 0.0 cm（定点保持限速 40 mm/s）
  ④ PARSE_ORDER    解析巡视顺序
  ④                顺序 = 书房 -> 厨房 -> 卧室（字母 BCD）
  ⑤ ENV_ANNOUNCE   读取客厅光照/温度/湿度并播报
  ⑤                当前客厅环境：光照强度 54.0lx，温度 21.5°C，湿度 55.7%
  ⑥ FACE_GATE      到 A 点做身份核验
  ⑥                核验通过（采样 12 轮，有脸 12 轮，命中主人 12 轮，比例 100%）
  ⑦a VOICE_PREP    停相机 + 后台启动语音引擎（为 A 点交互做准备）
  ⑦ ROOMS          按顺序巡视：书房 -> 厨房 -> 卧室
  ⑦ 书房             漏水检测并播报
  ⑦ 书房             漏水：正常（abnormal=False）
  ⑦                在书房停留 3 s
  ⑦ 厨房             读烟雾/CO2 状态 → 开传感器电源 → 开风扇
  ⑦ 厨房             烟雾状态：正常
  ⑦                厨房 倒车出房间（0.5 m）
  ⑦ 卧室             开电灯 + 开窗帘
  ⑦                在卧室停留 3 s
  ⑦b VOICE         语音交互（三轮自由控制，说完再开麦）
  ⑦b               已回到 A 点，开始语音交互
  ⑧ RETURN_START   返回起点（分两段：先回客厅，再回起点）
  ⑧                用固定起点航点（0.00, 0.00）
  ⑧                已回到客厅，继续回起点
  ⑧                关闭厨房风扇和卧室电灯
  ⑧                  fan_kitchen    off -> True
  ⑧                  light_bedroom  off -> True
```

—— 报告结束 ——